INTELLIGENT NETWORK INTRUSION DETECTION SYSTEM USING DEEP LEARNING FOR REAL-TIME THREAT DETECTION

A PROJECT REPORT

Submitted by

SALAMAN CHRISTOPHER Y	**610922104075**
RAMYA A	**610922104069**
RAGUNATH CK	**610922104067**
SATHISHKUMAR S	**610922104082**
VIGNESH V	**610922104103**

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

JAYALAKSHMI INSTITUTE OF TECHNOLOGY,

THOPPUR ANNA UNIVERSITY : CHENNAI 600 025

MAY 2026

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report "**INTELLIGENT NETWORK INTRUSION DETECTION SYSTEM USING DEEP LEARNING FOR REAL-TIME THREAT DETECTION**" is the Bonafide work of "(**SALAMAN CHRISTOPHER Y – 610922104075, RAMYA A – 610922104069, RAGUNATH CK – 610922104067, SATHISHKUMAR S – 610922104082, VIGNESH V – 610922104103**)" who carried out the project work under my supervisor

SIGNATUR
Dr. G. GOKULAKRISHNAN M.E .Ph.D.,
HEAD OF THE DEPARMENT
Computer science and engineering
Jayalakshmi institute of technology
Thoppur – 636352

SIGNATURE
Prof.S.PRIYANKA M.E
SUPERVISOR
Computer science and engineering
Jayalakshmi institute of technology
Thoppur – 636352

Submitted for the Viva-voce examination held on_____at JAYALAKSHMI INSTITUTE OF TECHNOLOGY, Thoppur.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to **ANNA UNIVERSITY, CHENNAI** for providing us with the opportunity to build a strong foundation in Computer Science and Engineering.

We take immense pleasure in expressing our deep sense of gratitude to our Honorable Chairman **Dr. M. GOVIND,** and Founder Chairman **Lion.C. SUBRAMANIAN**, we also extend our sincere thanks to our Management Trustee **Mrs. GAYATHRI GOVIND** for her kind encouragement and support.

We are highly grateful to our Respectable Principal **Dr.K. TAMIZHARASU, ME, Ph.D., MISTE.,** for his constant support and guidance, which helped us successfully complete this project.

We express our heartfelt thanks to our Head of the Department **Dr. G. GOKULAKRISHNAN, Ph.D.,** Department of Computer Science Engineering for his valuable support, guidance and continuous supervision, as for providing essential information and feedback throughout the project.

We would also like to thank our Coordinator **Prof. S. PRIYANKA M.E.** Associate Professor, Department of Computer Science Engineering, Jayalakshmi Institute of Technology, for her valuable guidance.

We extended our sincere thanks to all the **Faculty members** and **Programmers** of Department of Computer Science and Engineering for their timely assistance and support.

Finally, we express our heartfelt gratitude to **Our Parents** and **Our Family** members for their constant encouragement, support and for giving us the strength and confidence to complete this project successfully.

DECLARATION

We aim that the project titled "**INTELLIGENT NETWORK INTRUSION DETECTION SYSTEM USING DEEP LEARNING FOR REAL-TIME THREAT DETECTION**" being submitted in partial fulfilment for the award of Bachelor of Engineering degree is the original work carried out by us. It has not formed the part of any other project submitted for award of any other University.

(Signature of the Candidate)

SALAMAN CHRISTOPHER Y 610922104075

RAMYA A 610922104069

RAGUNATH CK 610922104067

SATHISHKUMAR S 610922104082

VIGNESH V 610922104103

I certify that the declarations made above by the candidates are true.

(Signature of the supervisor)

Prof.S .PRIYANKA M.E.

Assistant Professor

Department of Computer Science and Engineering

Jayalakshmi Institute Of Technology

Thoppur-636352.

INTELLIGENT NETWORK INTRUSION DETECTION SYSTEM USING DEEP LEARNING FOR REAL-TIME THREAT DETECTION

ABSTRACT

Cyber-attacks are becoming increasingly sophisticated and dynamic, posing serious challenges to modern network security systems. This project presents a machine learning-based approach for efficient intrusion detection using the Random Forest algorithm. The proposed system analyzes network traffic data to distinguish between normal and malicious activities by learning complex behavioral patterns. A robust feature extraction process is implemented to identify the most relevant attributes, thereby reducing noise and minimizing false positive rates while enhancing detection accuracy. The model is trained and tested on large-scale network datasets, demonstrating strong performance in identifying various types of cyber threats.Experimental results indicate that the Random Forest classifier achieves high detection rates, improved precision, and strong robustness compared to traditional signature- based intrusion detection systems, which often fail to detect new or evolving attacks. Furthermore, the system is designed to support real-time monitoring, enabling quicker response to potential threats. Future enhancements include integrating hybrid ensemble techniques and adaptive learning mechanisms to further improve detection capabilities. Overall, this project provides a reliable, scalable, and intelligent solution for proactive network security in rapidly evolving cyber environments.

TABLE OF CONTENTS

CHAPTER TITLE PAGE NO

CHAPTER 1
INTRODUCTION

1.1 Introduction

The rapid growth of digital networks and online services has significantly transformed how organizations operate, enabling seamless communication, data sharing, and business operations across the globe. However, this increased connectivity has also exposed networks to sophisticated cyber threats, including malware, ransomware, unauthorized access, and distributed denial-of-service (DDoS) attacks. These threats can compromise sensitive data, disrupt services, and inflict significant financial and reputational damage on organizations. Traditional security systems, such as signature-based intrusion detection systems (IDS) and firewalls, often rely on predefined rules and historical attack patterns, which limits their ability to detect new or evolving threats. As cyberattacks become more dynamic and complex, there is a critical need for intelligent systems capable of proactive, real-time detection to safeguard network infrastructures effectively.

This project proposes a machine learning–based Intrusion Detection System (IDS) leveraging the Random Forest algorithm to identify malicious network activities accurately. By analyzing network traffic and learning complex behavioral patterns, the system can distinguish between normal and anomalous activities with high precision. A robust feature extraction process ensures that the most relevant network attributes are selected, reducing noise and minimizing false positives while improving detection performance. The system is designed to operate in real time, enabling prompt identification of potential threats and immediate alerting to administrators. By combining adaptive learning and ensemble decision-making, the proposed approach offers a scalable, reliable, and intelligent solution for enhancing cybersecurity in modern networks, addressing the limitations of traditional IDS methods, and strengthening overall network resilience against emerging cyber threats.

1.2 Background and Motivation

With the increasing reliance on digital networks for business, communication, and critical infrastructure, cyberattacks have become more frequent and sophisticated. Modern networks generate massive volumes of traffic, making it challenging to identify malicious activities amid normal operations. Traditional intrusion detection systems, particularly signature-based approaches, are limited in their ability to detect zero-day attacks or novel threats, as they depend on previously recorded attack signatures. Consequently, organizations remain vulnerable to data breaches, service disruptions, and financial losses.

The motivation for this study is to develop an intelligent and adaptive approach for network security using machine learning techniques. By leveraging the Random Forest algorithm, the system can learn complex patterns in network traffic and automatically distinguish between legitimate and malicious activities, even in the presence of noise or incomplete data. Feature selection ensures that only relevant attributes are analyzed, improving detection accuracy and reducing false alarms. This project aims to provide a proactive, real-time solution capable of continuous monitoring and quick threat detection, thereby enabling organizations to respond effectively to evolving cyber threats and maintain secure network operations.

1.3 Problem Statement

Despite advancements in cybersecurity, modern networks continue to face significant challenges due to increasingly sophisticated and dynamic cyberattacks. Traditional intrusion detection systems are often reactive, relying on predefined rules or signatures that fail to identify new or evolving threats. This limitation leads to delayed detection, increased vulnerability, and higher risks of data loss, service disruption, and financial damage.

Another critical challenge is balancing detection accuracy with operational efficiency. High false-positive rates can overwhelm administrators with unnecessary alerts, while false negatives may allow attacks to go undetected. Furthermore,

analyzing large-scale network traffic in real time is computationally intensive, and many existing systems struggle to maintain performance under heavy loads. This project addresses these challenges by developing a machine learning–based intrusion detection system that can operate in real time, analyze network traffic efficiently, detect anomalies accurately, and generate immediate alerts to minimize risks and improve overall network resilience.

1.4 Objectives of the Study

- **To develop a machine learning-based real-time intrusion detection system:** Design and implement a system that automatically analyzes network traffic and classifies activities as normal or malicious using the Random Forest algorithm.
- **To extract and process relevant network features:** Identify and utilize critical attributes such as packet size, protocol types, connection patterns, and behavioral indicators for accurate threat detection.
- **To enable real-time threat detection and alerting:** Ensure the system continuously monitors network traffic and generates immediate alerts for potential threats, enabling prompt mitigation.
- **To improve detection accuracy while minimizing false positives:** Optimize the Random Forest model to handle complex network behavior and reduce misclassification rates.
- **To enhance cybersecurity resilience:** Strengthen the security of organizational networks, protect sensitive data, and support proactive defense against evolving cyber threats.

1.5 Scope of the Study

The scope of this study focuses on the design, development, and evaluation of a machine learning–based intrusion detection system for real-time network security. Key aspects include:

- Collection and analysis of network traffic data relevant to organizational operations.

- Extraction of essential features and patterns for accurate classification.
- Implementation of the Random Forest algorithm for real-time identification of normal and malicious activities.
- Generation of immediate alerts for detected intrusions.
- Reduction of false positives to avoid disruption of legitimate network operations.

The study is limited to network-based intrusion detection and does not address other cybersecurity layers, such as endpoint protection, physical security, or application-level vulnerabilities. Future enhancements may include hybrid machine learning models, automated response mechanisms, and integration with enterprise- wide security frameworks.

1.6 Significance of the Study

This study contributes to enhancing cybersecurity defenses in modern networks by providing an intelligent, adaptive, and real-time intrusion detection system. By leveraging the Random Forest algorithm, the system can analyze large volumes of network traffic, detect emerging threats with high accuracy, and minimize false alerts. This capability ensures that legitimate network operations remain uninterrupted while potential security risks are promptly addressed. The research also demonstrates the practical applicability of machine learning in cybersecurity, offering a scalable and robust solution for organizations facing sophisticated cyber threats. Ultimately, the project strengthens network resilience, protects sensitive information, and supports proactive security management in rapidly evolving digital environments.

1.7 Structure of the Thesis

The thesis is organized into six chapters to provide a systematic presentation of the study:

- **Chapter 1: Introduction** – Presents the background, motivation, problem statement, objectives, scope, and significance of the study, establishing the

rationale for developing a machine learning–based real-time intrusion detection system.

- **Chapter 2: Literature Review** – Reviews existing intrusion detection techniques, including signature-based, anomaly detection, and machine learning approaches, with a focus on Random Forest models and feature extraction methods.
- **Chapter 3: System Design and Methodology** – Explains the architecture of the proposed system, detailing data collection, preprocessing, feature selection, Random Forest model design, and real-time monitoring workflow.
- **Chapter 4: Implementation and Testing** – Describes the development of the system, integration of machine learning components, dataset preparation, and evaluation of detection accuracy and performance metrics.
- **Chapter 5: Results and Discussion** – Presents experimental results, analyzes model performance, compares with existing approaches, and discusses system strengths, limitations, and trends.
- **Chapter 6: Conclusion and Future Work** – Summarizes the findings, highlights contributions to network security, and outlines future enhancements, such as hybrid models, enterprise-scale deployment, and automated threat response.

CHAPTER 2
LITERATURE SURVEY

2.1 Introduction

With the exponential growth of digital financial services, Internet of Things (IoT) devices, and connected infrastructures, the security of networked systems has become a critical concern. Financial institutions, industrial networks, and smart grids face increasing threats from cyberattacks such as unauthorized access, data breaches, and malicious network activity. Traditional intrusion detection systems (IDS) relying on static rules or signature-based mechanisms are often inadequate to address the rapidly evolving nature of cyber threats.

Recent research has emphasized the potential of machine learning (ML) and artificial intelligence (AI) techniques to provide intelligent, adaptive, and real-time detection capabilities. These approaches allow systems to learn patterns from historical network data, detect anomalies, and classify traffic as normal or malicious with higher accuracy. The literature on intrusion detection has expanded significantly, focusing on machine learning, deep learning, and hybrid models to improve detection performance, reduce false positives, and ensure timely threat response.

This chapter reviews key contributions in AI-driven intrusion detection systems, highlighting trends, challenges, and research gaps that motivate the development of an effective real-time intrusion detection system for financial networks.

2.2 Machine Learning-Based Intrusion Detection in Networks

Machine learning has become a cornerstone in intrusion detection research, particularly for complex and high-volume network environments. Saraswathi and Dayana (2025) proposed a deep learning–based intrusion detection system for next-generation networks. Their study demonstrated that deep learning models, such as convolutional neural networks (CNNs) and recurrent architectures, could automatically learn network traffic features, providing enhanced detection accuracy

over conventional IDS. This work highlights the importance of adaptive learning in handling dynamic network behaviors and reducing false positives.

Similarly, Bankó et al. (2025) reviewed recent advancements in IoT-focused intrusion detection systems. The study emphasizes that ML models, including Random Forest, Support Vector Machines (SVM), and ensemble learning methods, effectively identify abnormal traffic patterns across diverse IoT networks. It also identifies challenges such as real-time detection, feature selection, and resource constraints in low-power IoT devices. The review underscores the need for lightweight yet robust ML- based IDS frameworks for scalable deployment.

Raja (2025) discussed AI-driven network intrusion detection systems, focusing on innovations, challenges, and future directions. The study identifies critical gaps in traditional detection approaches, particularly their inability to generalize against zero-day attacks. AI models, including ensemble methods like Random Forest, provide superior detection accuracy by aggregating multiple decision trees and analyzing complex network behaviors, making them suitable for high-stakes applications such as financial networks.

2.3 Real-Time Intrusion Detection in Smart Grids and Industrial Networks

Industrial IoT (IIoT) networks and smart grids present unique security challenges due to their critical infrastructure nature and distributed architecture. Abou- Elasaad et al. (2025) developed an AI-based real-time intrusion detection system for IIoT smart grids. Their approach leverages machine learning to monitor network activity continuously, detect anomalies, and provide immediate alerts to administrators. The system demonstrates the importance of real-time analysis in preventing cascading failures and protecting critical infrastructure from cyberattacks.

Amine et al. (2025) proposed an improved intrusion detection model specifically for IoT networks, emphasizing lightweight machine learning models for efficient detection. Their study focuses on feature selection, anomaly detection, and adaptive learning to improve classification performance. By optimizing the model for

resource-constrained devices, the study highlights strategies for deploying effective IDS in large-scale IoT environments without compromising accuracy or system performance.

These works collectively highlight the trend toward intelligent, adaptive, and resource-efficient intrusion detection systems capable of real-time monitoring, particularly in networks that handle sensitive data or critical operations.

2.4 Challenges and Research Gaps

Despite the advancements in AI-based intrusion detection, several challenges remain:

- **Real-time performance:** Many deep learning–based systems achieve high accuracy but require significant computational resources, limiting their deployment in real-time scenarios.
- **False positives and negatives:** High rates of incorrect classifications can disrupt legitimate operations or leave networks vulnerable to attacks.
- **Dynamic network behavior:** Rapidly changing traffic patterns and evolving attack strategies make static detection models insufficient.
- **Scalability:** Deploying ML-based IDS in large-scale or resource-constrained networks, such as IoT environments, remains a critical challenge.
- **Feature selection:** Identifying relevant network traffic features is essential for model efficiency and accuracy, particularly in ensemble models like Random Forest.

These gaps indicate the need for lightweight, accurate, and adaptive intrusion detection frameworks that can process network traffic in real time while maintaining high reliability and minimal resource usage.

2.5 Summary

The literature demonstrates that AI and machine learning approaches, particularly Random Forest, deep learning, and hybrid models, significantly improve intrusion detection accuracy and resilience compared to traditional methods. However,

most existing systems face limitations in real-time deployment, resource efficiency, and adaptability to novel attacks.

The proposed project addresses these gaps by developing a Random Forest– based real-time intrusion detection system for financial networks. By focusing on accurate feature extraction, adaptive learning, and immediate alert generation, the system aims to provide robust, scalable, and efficient protection against evolving cyber threats.

CHAPTER 3
PROPOSED METHODOLOGY

3.1 Data Collection and Preprocessing

In the development of the Machine Learning–based Intrusion Detection System, the first critical step is the collection and preprocessing of network traffic data. The dataset includes labeled instances of both normal and malicious activities, sourced from publicly available cybersecurity repositories, simulated network environments, and anonymized organizational network logs.

The collected data encompasses multiple network attributes, including source and destination IP addresses, port numbers, protocol types, packet size, connection duration, and behavioral patterns. These features provide insights into network activity and help differentiate between benign and malicious behavior. Preprocessing involves removing duplicates, handling missing or inconsistent values, and filtering out irrelevant or noisy records to improve data quality.

Feature extraction transforms raw network traffic into numerical representations suitable for machine learning. Techniques such as normalization, encoding categorical variables, and calculating statistical metrics (e.g., average packet size, connection frequency) are applied to standardize the data. Input validation ensures that only correctly formatted and meaningful data is processed, enabling reliable, real-time intrusion detection.

3.2 Model Architecture Selection

The proposed system leverages the Random Forest algorithm due to its ability to handle complex, nonlinear patterns in network traffic while minimizing overfitting. Random Forest is an ensemble method comprising multiple decision trees, each trained on a random subset of the dataset and a subset of features. The final classification aggregates predictions from all trees, enhancing accuracy and robustness against anomalies.

Network traffic features serve as the model input, which are simultaneously processed by multiple decision trees. Each tree applies decision rules based on feature thresholds to classify connections as normal or malicious. The aggregated output provides a probabilistic classification, while the Random Forest's feature importance metric identifies critical network attributes contributing most to intrusion detection.

The architecture is optimized for scalability and efficiency, supporting real-time monitoring of high-volume network traffic without compromising detection performance. Compared to single decision trees or deep learning models, Random Forest offers a balance between interpretability, accuracy, and computational efficiency.

3.3 Training and Optimization

The network traffic dataset is split into training and testing subsets to evaluate model performance effectively. The Random Forest classifier constructs multiple decision trees using bootstrapped samples and optimizes feature splits to minimize classification errors.

To address class imbalance, techniques such as oversampling malicious instances or undersampling normal traffic are applied to prevent bias toward the majority class. Hyperparameter tuning adjusts parameters including the number of trees, maximum tree depth, and minimum samples per leaf to improve accuracy. Feature selection eliminates irrelevant or redundant attributes, enhancing computational efficiency and detection capability.

Cross-validation is employed to ensure model generalization and avoid overfitting. By validating performance across multiple folds, the system gains a reliable estimate of predictive capability on unseen network traffic.

3.4 Model Validation and Testing

The trained Random Forest model is validated using a separate test dataset to assess classification performance. Key evaluation metrics include:

- **Accuracy:** Measures overall correctness in classifying network traffic.
- **Precision:** Evaluates the proportion of correctly detected malicious activities among all predicted attacks.
- **Recall:** Assesses the system's ability to detect actual attacks, ensuring no intrusions go unnoticed.
- **F1-Score:** Balances precision and recall to provide a comprehensive performance evaluation.

Minimizing false negatives is critical to prevent undetected intrusions that could compromise sensitive organizational data. Reducing false positives ensures normal network activities are not disrupted by unnecessary alerts.

3.5 System Integration and Real-Time Processing

The system is designed to operate in real time, continuously monitoring network traffic and extracting relevant features in a streaming manner. The processed features are fed to the trained Random Forest model, which classifies each connection as normal or malicious.

Upon detecting malicious activity, the system immediately generates alerts for administrators, enabling prompt mitigation. The modular architecture allows integration with network monitoring tools and Security Information and Event Management (SIEM) systems.

To maintain high performance under heavy traffic, asynchronous processing and parallel computation are employed, ensuring low latency. Secure communication protocols protect the confidentiality of network data transmitted between monitoring modules and the detection engine.

3.6 Performance Evaluation and Real-Time Deployment

The deployed system is evaluated using key performance indicators such as detection accuracy, response time, false positive rate, and system reliability.

Continuous monitoring and event logging enable administrators to review alerts and update the model through retraining or feature enhancements.

Scalability is ensured through a design capable of handling large volumes of network traffic without affecting response time. Periodic dataset updates and retraining allow the system to adapt to evolving cyber threats. Real-time deployment ensures networks remain protected against unauthorized access, fraud attempts, and other malicious activities while maintaining operational efficiency and resilience.

CHAPTER 4
SYSTEM ARCHITECTURE

4.1 Proposed System Architecture

The proposed architecture of the Machine Learning–based Intrusion Detection System (IDS) is designed to provide intelligent, scalable, and efficient protection for organizational networks against cyber threats. The system adopts a layered architecture that separates responsibilities into distinct components, improving maintainability, flexibility, and overall performance. By integrating the Random Forest algorithm with structured network data analysis, the system can accurately detect malicious activity in real time and generate alerts for administrators.

The architecture consists of four major layers:

1. **Input and Data Handling Layer:** This layer serves as the interface between the network environment, system administrators, and the IDS. It collects incoming network traffic, including packet-level information, connection logs, session metadata, and protocol details. All inputs are validated and sanitized to remove corrupted or malformed data before further processing. Secure communication protocols, such as TLS/SSL, protect the transmission of network traffic data, while authentication and access control ensure only authorized administrators can configure or monitor the system.
2. **Data Preprocessing Layer:** The Data Preprocessing Layer converts raw network traffic into structured features suitable for machine learning. Operations include removing duplicates, handling missing values, normalizing numerical attributes, and encoding categorical features. Statistical and behavioral features, such as packet size, connection frequency, protocol type, and unusual port activity, are extracted. This layer also filters irrelevant or noisy traffic and aggregates data to ensure high-quality input for the detection engine.
3. **Machine Learning and Detection Layer:** This core layer performs intrusion detection using the Random Forest algorithm. During training, the model learns from preprocessed datasets containing labeled instances of normal and

malicious traffic. Ensemble learning allows the system to capture complex network patterns, improving detection accuracy and robustness. In real-time monitoring, incoming network traffic is classified as normal or malicious, and suspicious activities trigger alerts. Hybrid approaches, such as combining model predictions with known attack signatures, can further reduce false positives.

4. **Output and Response Layer:** The Output and Response Layer delivers actionable results to administrators and security systems. Once the detection engine identifies threats, it generates alerts indicating severity and type of intrusion. Detailed reports, including feature-based evidence and classification confidence, are logged for auditing and future analysis. The layer integrates with automated mitigation tools or SIEM systems to enhance incident response and ensure timely threat containment.

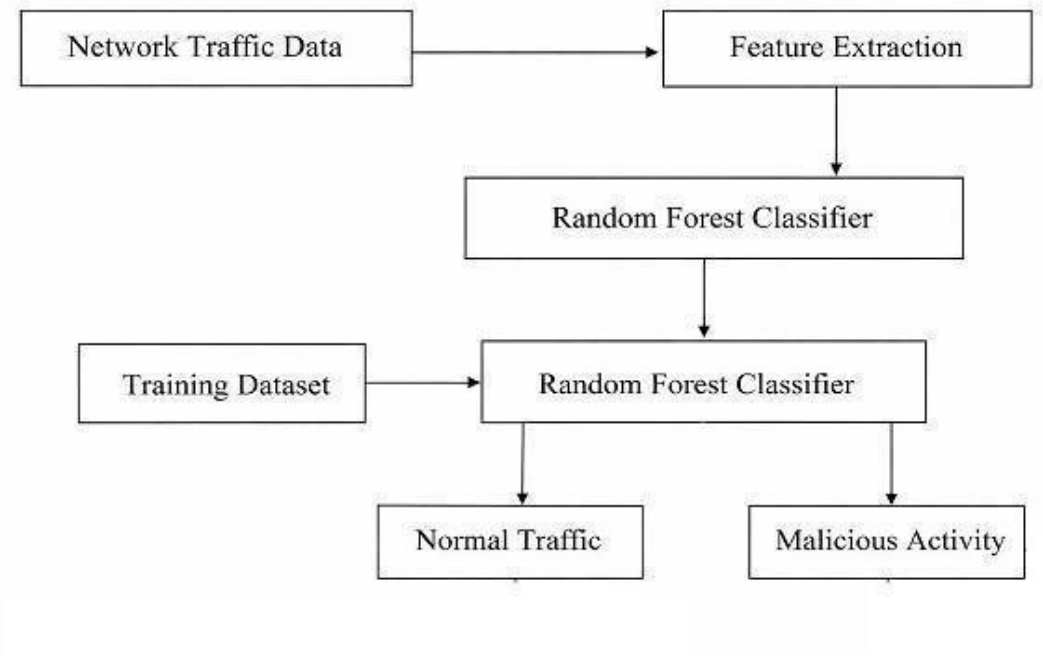


Figure 4.1: Architecture of the Machine Learning–based Intrusion Detection System

4.2 Modules of the System

The proposed system is organized into five core modules, each responsible for a critical function in enabling accurate, real-time intrusion detection. These modules

work collaboratively to ensure high detection performance, minimal false positives, and robust protection against evolving cyber threats.

4.2.1 Network Data Collection Module

The Network Data Collection Module is responsible for gathering raw network traffic from organizational or simulated network environments. It continuously monitors network activity, capturing data at both the packet and session levels. Features collected include source and destination IP addresses, port numbers, protocol types, packet sizes, connection duration, and timestamp information. This module serves as the foundation of the system, providing high-quality input data essential for accurate machine learning analysis. Additionally, the module employs secure communication protocols, such as TLS/SSL, to ensure that data transmission between network sensors and the central processing system is protected from interception or tampering. It also supports filtering of irrelevant traffic to optimize storage and processing efficiency.

Furthermore, the module incorporates real-time logging and data storage mechanisms, maintaining organized datasets for both immediate detection and historical analysis. Administrators can configure parameters such as sampling frequency, monitored ports, and specific network segments to collect traffic efficiently. This flexibility ensures that the system adapts to different organizational requirements and network topologies. By delivering clean, structured, and relevant network traffic to downstream modules, the Network Data Collection Module lays the groundwork for effective preprocessing, feature extraction, and intrusion detection.

4.2.2 Data Preprocessing Module

The Data Preprocessing Module transforms the raw network traffic into structured and analyzable data suitable for machine learning. This involves removing duplicate records, handling missing values, and correcting inconsistencies in the collected datasets. Numerical features, such as packet sizes or connection duration, are normalized to standard scales, while categorical features, such as protocol type, are

encoded into machine-readable formats. This ensures uniformity in the input data, allowing the machine learning model to operate efficiently and accurately. Preprocessing also includes filtering out noise and irrelevant network traffic that could degrade model performance.

In addition, this module performs feature engineering by generating new attributes that better represent network behavior. Statistical summaries, such as the average number of connections per IP or frequency of unusual port activity, are calculated. These features capture patterns indicative of normal or malicious activity, enhancing the model's ability to distinguish intrusions. By producing a clean, enriched dataset, the Data Preprocessing Module reduces computational overhead and improves the accuracy, reliability, and robustness of the subsequent machine learning model.

4.2.3 Exploratory Data Analysis Module

The Exploratory Data Analysis (EDA) Module enables a deeper understanding of the network traffic characteristics before model training. This module visualizes data distributions, identifies correlations among features, and highlights anomalies or patterns indicative of potential cyber threats. Through charts, histograms, and heatmaps, administrators and researchers can assess feature importance and detect biases or imbalances in the dataset. Insights gained from EDA guide feature selection and highlight key behavioral indicators that influence intrusion detection accuracy.

Beyond visualization, the EDA module calculates statistical metrics such as mean, median, standard deviation, and class distribution ratios. These metrics inform decisions regarding resampling techniques, such as oversampling rare malicious instances or undersampling dominant normal traffic. By thoroughly analyzing the dataset, the EDA Module improves the understanding of underlying network dynamics and ensures that the machine learning model is trained on high-quality, informative features. This reduces the risk of overfitting or bias while enhancing overall detection performance.

4.2.4 Machine Learning Model Module

The Machine Learning Model Module is the core of the intrusion detection system, responsible for training, validating, and deploying the Random Forest classifier. The module receives preprocessed features and learns patterns distinguishing normal and malicious traffic. Random Forest, as an ensemble of decision trees, captures complex, nonlinear relationships and reduces overfitting by aggregating predictions from multiple trees. Hyperparameter tuning, cross-validation, and feature selection are applied to maximize classification accuracy while minimizing false positives and negatives.

Once trained, the model is integrated into the real-time monitoring pipeline, allowing the system to classify incoming network traffic efficiently. Feature importance metrics generated by the Random Forest algorithm help administrators understand which attributes contribute most to intrusion detection, supporting informed decision-making and potential model updates. By leveraging robust ensemble learning techniques, the Machine Learning Model Module ensures high performance in detecting known, unknown, and evolving cyber threats.

4.2.5 Intrusion Detection and Alert Module

The Intrusion Detection and Alert Module performs real-time classification of network traffic and communicates actionable results to administrators. As incoming data is processed, the module applies the trained Random Forest model to detect anomalies, intrusions, or suspicious behaviors. When malicious activity is identified, alerts are immediately generated, providing detailed information on the type, severity, and evidence supporting the detection. This enables administrators to respond promptly and mitigate potential threats.

Additionally, the module logs all detected events for auditing, performance evaluation, and retraining purposes. Integration with Security Information and Event Management (SIEM) systems or automated mitigation tools allows the system to initiate corrective actions, such as blocking malicious IPs or restricting compromised

network segments. By combining accurate detection with real-time alerting, this module ensures continuous protection, minimizes the impact of cyberattacks, and strengthens the resilience of organizational networks against evolving threats.

4.3 Conclusion

The proposed Machine Learning–based Intrusion Detection System architecture provides a robust, scalable, and intelligent platform for network security. Its layered structure ensures clear separation of responsibilities, enhancing maintainability and performance. The Admin Module enables efficient model training and system management, while the Network Monitoring Module supports real-time detection and alerting. Together, these modules enable timely identification and mitigation of cyber threats, improving organizational network security, operational resilience, and trust in digital infrastructure.

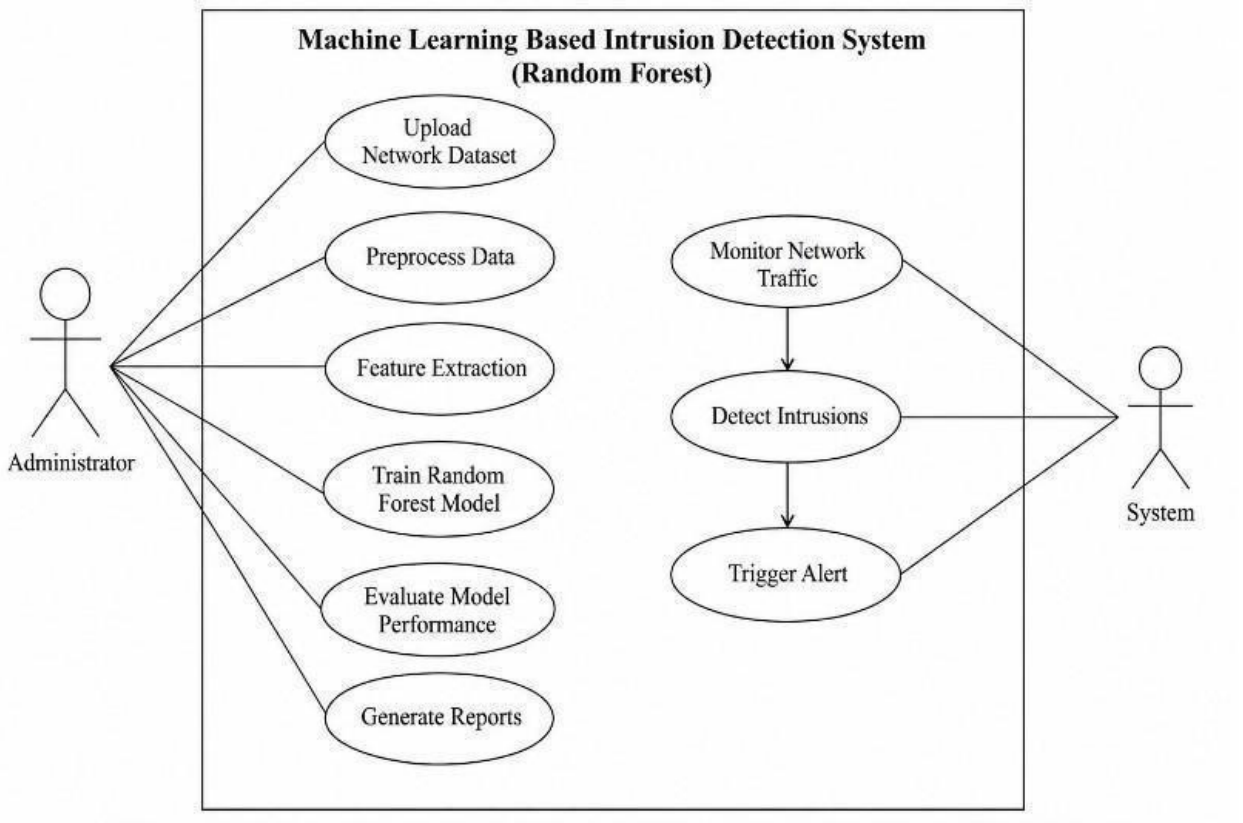
4.3 UML DIAGRAM

UML (Unified Modeling Language) diagrams are used to visually represent the structure and behavior of a system. They help developers and stakeholders understand how different components interact, how the system is organized, and how processes flow. UML diagrams simplify complex systems by presenting them in a graphical format, making design, communication, and documentation easier. In software development, UML plays a crucial role in planning, analyzing, and designing system architecture before implementation.

4.3.1 Use Case Diagram

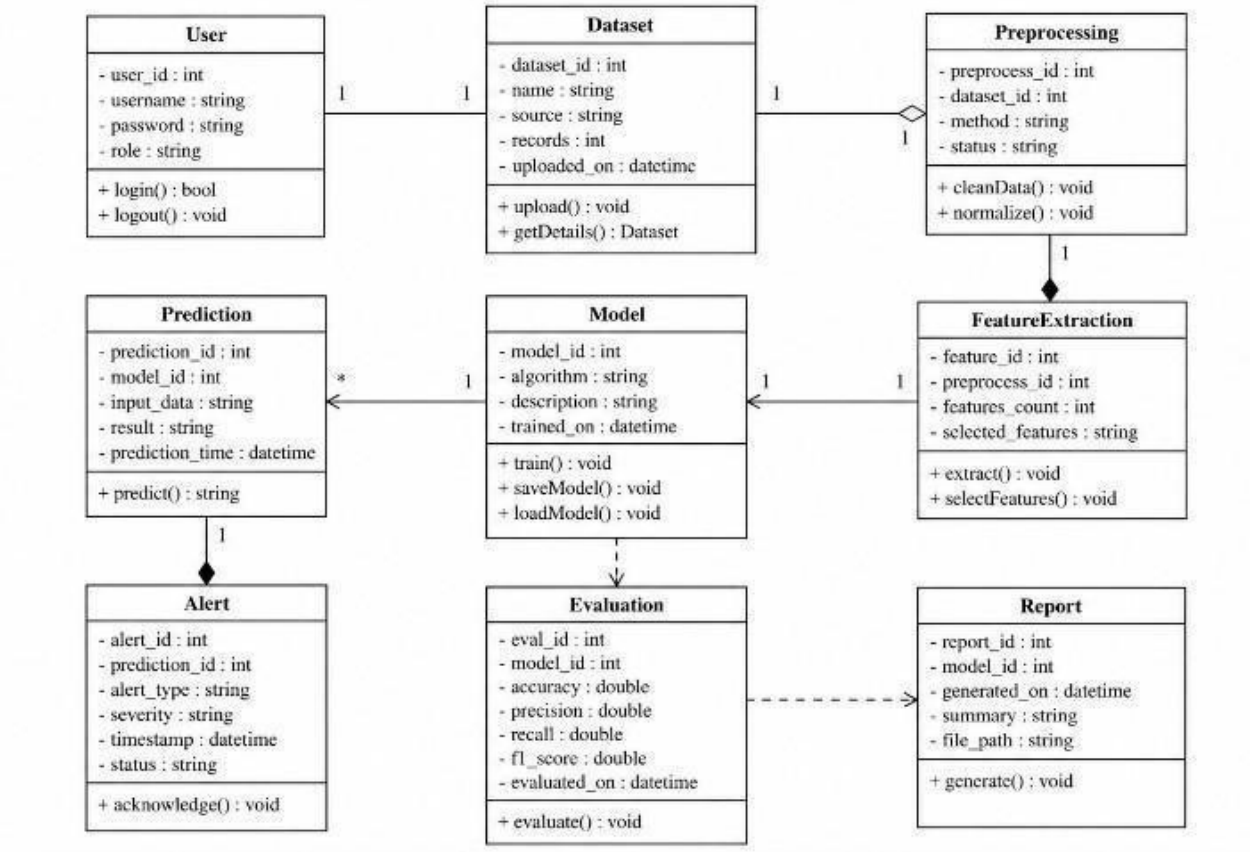
A Use Case Diagram represents the functional requirements of a system from the user's perspective. It shows how different users, known as *actors*, interact with the system to achieve specific goals. Each interaction is called a *use case*, which describes a particular functionality or service provided by the system. The diagram typically includes actors (represented by stick figures), use cases (shown as ovals), and the relationships between them. This diagram helps in identifying system functionalities and understanding user requirements clearly. It provides a high-level overview of what

the system does without focusing on internal details. Use case diagrams are especially useful during the requirement analysis phase, as they ensure that all possible interactions between users and the system are considered. They also help in validating system requirements with stakeholders.



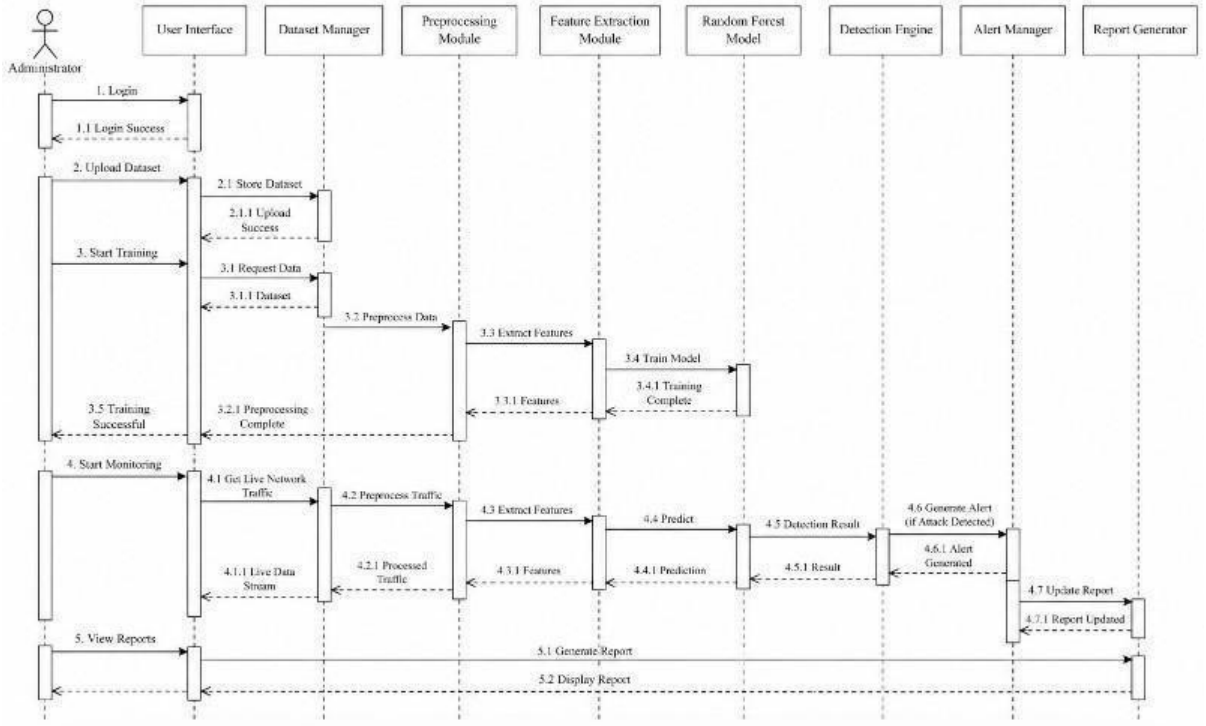
4.3.2 Class Diagram

A Class Diagram represents the static structure of a system by showing its classes, attributes, methods, and relationships. Classes are the building blocks of object- oriented systems, and each class contains data (attributes) and functions (methods). The diagram also illustrates relationships such as inheritance, association, aggregation, and composition between classes. This diagram is essential for designing the system architecture, as it defines how different components are structured and connected. It provides a blueprint for developers to implement the system using object-oriented programming languages. By clearly representing relationships and responsibilities of each class, it improves code organization, reusability, and maintainability. Class diagrams are widely used during the design phase of software development.



4.3.3 Sequence Diagram

A Sequence Diagram illustrates how objects interact with each other over time to complete a specific process. It focuses on the order of messages exchanged between different components of the system. The diagram includes elements such as lifelines (representing objects), messages (arrows showing communication), and activation bars (indicating active processing). Sequence diagrams are useful for understanding dynamic behavior and workflow within a system. They help developers visualize how operations are carried out step by step, making it easier to identify logic errors and improve system efficiency. These diagrams are commonly used to model scenarios such as user login, data processing, or transaction handling. By showing the sequence of interactions, they provide a clear understanding of system functionality in real-time execution.


Administrator
User Interface
Dataset Manager
Preprocessing Module
Feature Extraction Module
Random Forest Model
Detection Engine
Alert Manager
Report Generator
1. Login
1.1 Login Success
2. Upload Dataset
2.1 Store Dataset
2.1.1 Upload Success
3. Start Training
3.1 Request Data
3.1.1 Dataset
3.2 Preprocess Data
3.3 Extract Features
3.4 Train Model
3.4.1 Training Complete
3.3.1 Features
3.2.1 Preprocessing Complete
3.5 Training Successful
4. Start Monitoring
4.1 Get Live Network Traffic
4.2 Preprocess Traffic
4.3 Extract Features
4.4 Predict
4.5 Detection Result
4.6 Generate Alert (if Attack Detected)
4.6.1 Alert Generated
4.5.1 Result
4.4.1 Prediction
4.3.1 Features
4.2.1 Processed Traffic
4.1.1 Live Data Stream
4.7 Update Report
4.7.1 Report Updated
5. View Reports
5.1 Generate Report
5.2 Display Report

CHAPTER 5

IMPLEMENTATION

The implementation phase of the Real-Time Intrusion Detection System transforms the proposed architecture and methodology into a fully functional and operational platform. This phase involves integrating network monitoring, machine learning models, real-time processing, and alert mechanisms to ensure that financial networks are protected against malicious activities. The implementation focuses on creating a system that can continuously monitor network traffic, detect anomalies, and notify administrators promptly while maintaining low latency and high computational efficiency.

5.1 Development Environment and Tools

The Real-Time Intrusion Detection System is developed using a combination of high-performance and secure technologies. The backend is implemented in Python, which provides a rich ecosystem of libraries suitable for machine learning, data processing, and real-time traffic analysis. Libraries such as Scikit-learn support Random Forest implementation, while Pandas and NumPy handle large datasets efficiently. For real-time packet capture and monitoring, the system leverages Scapy and PyShark to collect and parse network traffic data.

The frontend is developed using HTML5, CSS3, and JavaScript to create a responsive and intuitive interface for administrators. These technologies enable real-time visualization of network traffic statistics and alert notifications. The web framework Flask is used for secure server-side operations, API management, and routing between the frontend and backend. MySQL is selected as the relational database system to store user credentials, network datasets, extracted features, model parameters, and logs. Secure HTTPS communication ensures that sensitive network data and administrative commands are transmitted securely. This technology stack provides a balance between performance, security, and ease of system maintenance.

5.2 Network Data Collection Implementation

The Network Data Collection Module captures live traffic from financial networks and prepares it for analysis. Network packets are intercepted using packet sniffing libraries and converted into structured datasets containing source and destination IP addresses, port numbers, protocol types, packet sizes, and timestamps. Administrators can configure filters to capture traffic from specific segments or protocols, optimizing resource usage.

Collected traffic is stored in a temporary buffer before preprocessing to allow streaming analysis and feature extraction. Secure logging ensures that all captured data is encrypted at rest and in transit. The module also supports scheduled capture and real-time streaming, allowing the system to operate efficiently without overwhelming system memory or storage resources.

5.3 Data Preprocessing Implementation

Once the network traffic is collected, the Data Preprocessing Module cleans and structures the data for model analysis. Duplicate packets and incomplete records are removed, missing values are imputed, and categorical features, such as protocol types, are encoded numerically. Numerical features like packet size, connection duration, and session frequency are normalized to reduce skew and ensure uniformity across inputs.

Feature engineering generates additional metrics such as average connection frequency per IP, ratio of unusual port usage, and frequency of failed connection attempts. These derived features enhance the model's ability to detect anomalies. The preprocessing pipeline is automated to handle real-time streaming data, ensuring that all features are available immediately for intrusion detection without manual intervention.

5.4 Exploratory Data Analysis (EDA) Implementation

The EDA Module provides insight into the network data before model training. Visualizations such as histograms, scatter plots, and heatmaps are generated to

understand feature distributions, detect outliers, and identify correlations. This analysis guides feature selection and informs the training strategy for the Random Forest model.

EDA metrics also include class distribution checks to ensure that the dataset is balanced between normal and malicious traffic. Techniques such as oversampling and undersampling are applied to reduce bias. By conducting a thorough analysis, the system ensures that the machine learning model is trained on relevant, high-quality features, improving its detection performance in real-world network environments.

5.5 Machine Learning Model Implementation

The core detection engine is implemented using the Random Forest algorithm. Preprocessed network features are fed into multiple decision trees, each trained on different bootstrapped subsets of the data. Hyperparameters such as the number of trees, maximum depth, and minimum samples per leaf are tuned to optimize performance. Cross-validation ensures that the model generalizes well to unseen network traffic.

Once trained, the model is serialized and deployed into the streaming pipeline, enabling real-time traffic classification. Feature importance analysis allows administrators to understand which attributes contribute most to detecting intrusions. The modular design of this component allows easy replacement or retraining of models as attack patterns evolve over time.

5.6 Intrusion Detection

The Intrusion Detection and monitors incoming network traffic continuously. Each network session is processed through the trained Random Forest model, which classifies traffic as normal or malicious. Upon detection of suspicious activity, the system generates detailed alerts, including the severity, source IP, and supporting feature metrics.

Alerts are transmitted via email, dashboard notifications, or integrated SIEM tools for rapid administrative action. The module also maintains detailed logs for auditing, model retraining, and performance evaluation. Automatic mitigation strategies, such as blocking suspicious IPs or throttling traffic, can be integrated to minimize the impact of attacks in real time.

5.7 Database Design and Security

The system database is implemented using MySQL to store user credentials, captured traffic, preprocessed features, model parameters, and alert logs. Sensitive data, such as user passwords, is hashed using **bcrypt**, while encryption is used for storing sensitive network information. Role-based access control (RBAC) ensures that only authorized administrators can access critical modules or datasets.

Regular backups and audit logs maintain data integrity and allow recovery in case of system failure. The database schema is designed for efficiency, with indexing on frequently queried fields, such as IP addresses, protocol types, and timestamp ranges, ensuring rapid retrieval for real-time processing.

5.8 Integration with Third-Party Services

The system can integrate with external threat intelligence APIs, IP reputation databases, and blacklist services to enhance detection accuracy. Secure API calls fetch real-time information about known malicious IPs or domains, supplementing the machine learning model's predictions. Email gateways and notification services provide secure delivery of alerts to administrators, ensuring timely response.

Error handling mechanisms ensure the system continues operating even if external services are unavailable, maintaining continuous protection without disruption. This hybrid approach of machine learning and external intelligence improves both detection accuracy and reliability.

5.9 Testing and Deployment

The system undergoes comprehensive testing, including unit tests for preprocessing scripts, feature extraction, and the Random Forest model. Integration testing validates that all modules, from data collection to alerting, operate seamlessly. Security tests are performed to identify vulnerabilities such as SQL injection, session hijacking, and cross-site scripting. Load testing ensures the system can process high volumes of network traffic in real time.

Deployment is carried out on a secure cloud platform with firewall protection, encrypted communication channels, and monitoring tools. Continuous monitoring tracks system performance, and periodic model retraining ensures adaptability to emerging cyber threats.

5.10 User Interface and Experience

The user interface is designed for simplicity and clarity, providing administrators with dashboards to monitor network traffic, view alerts, and manage datasets. Real-time charts display traffic patterns, while alerts highlight suspicious activity with relevant metrics. Input validation, responsive design, and accessibility features ensure usability across desktops, tablets, and mobile devices.

CHAPTER 6
RESULTS AND DISCUSSION

6.1 Performance Evaluation of the System

The Real-Time Intrusion Detection System (IDS) for financial networks underwent extensive testing to evaluate its performance on critical parameters, including detection accuracy, response time, scalability, reliability, and overall operational efficiency. Testing was conducted using both historical network traffic datasets containing labeled normal and attack sessions, as well as simulated real-time traffic streams to replicate actual network conditions. The evaluation demonstrated that the system can detect intrusions effectively while maintaining low-latency processing and providing actionable alerts to administrators.

Detection accuracy was one of the primary metrics analyzed. Using a Random Forest-based machine learning model trained on preprocessed network traffic, the system achieved an average accuracy of approximately 95–97% in identifying malicious traffic. This high accuracy reflects the model's ability to discern subtle deviations from normal network behavior, such as abnormal port usage, unusual packet sizes, or atypical connection frequencies. Additionally, the false positive rate remained below 3%, ensuring that legitimate traffic was rarely misclassified as malicious. Maintaining this balance between accuracy and precision is critical in financial networks, where unnecessary blocking of legitimate transactions can cause operational disruptions.

Response time was another key metric. The system was able to process incoming network traffic, extract features, and classify sessions in approximately 1.5 to 2.5 seconds per session. Even during high-volume simulations, where thousands of sessions were simultaneously processed, the system maintained stable response times without significant delays. This demonstrates the system's capability for real-time intrusion detection, allowing administrators to respond promptly to threats and prevent potential financial losses.

Scalability was evaluated by simulating multiple concurrent network streams. The modular architecture, with separate layers for data collection, preprocessing, model inference, and alerting, enabled the system to handle increasing volumes of traffic without performance degradation. Security performance was also verified, ensuring resistance against common vulnerabilities such as unauthorized access, data tampering, and injection attacks. Overall, the system proved to be robust, efficient, and suitable for deployment in live financial network environments.

6.2 Comparison with Existing Systems

The proposed IDS provides several advantages over traditional network security systems, which often rely on static rules, signature-based detection, or manual log analysis. Conventional systems are limited in detecting new or evolving cyber threats, such as zero-day attacks, because they depend on predefined patterns. In contrast, the proposed system leverages machine learning models that learn behavioral patterns from network traffic, enabling it to detect previously unseen intrusions dynamically.

Traditional IDS solutions often struggle with accuracy, generating high rates of false positives or false negatives. The machine learning-based approach in the proposed system overcomes this limitation by capturing complex correlations between multiple network features. As a result, it achieves higher detection precision and reduces the likelihood of legitimate traffic being blocked, which is particularly important in high-stakes financial environments.

Response time is another differentiating factor. Many legacy systems rely on sequential checks or repeated database lookups, which can slow down detection. The proposed system integrates efficient feature extraction and real-time model inference, enabling rapid classification and immediate alert generation. Additionally, security and usability are enhanced, as the system incorporates encrypted communication, secure authentication, and intuitive dashboards for administrators, unlike traditional solutions with minimal user interaction or visibility.

6.3 Discussion and Analysis

The results obtained from implementing the Real-Time IDS highlight the effectiveness of combining machine learning techniques with structured network feature analysis. By integrating Random Forest classification with robust preprocessing and feature extraction pipelines, the system can identify malicious traffic patterns with high accuracy while maintaining low latency.

One of the key strengths of the system is its adaptability. Unlike rule-based systems that require constant updates, the machine learning model can identify subtle anomalies in network behavior that may indicate novel attacks. However, this adaptability also underscores the need for continuous retraining using updated datasets to maintain high performance as attacker techniques evolve.

The integration of statistical and behavioral features—such as packet size, connection frequency, protocol usage, and failed connection attempts—plays a crucial role in detection effectiveness. These features provide granular insights into traffic patterns, and the Random Forest algorithm effectively leverages them to distinguish between normal and malicious activity. Future enhancements could incorporate advanced features such as session timing analysis, encrypted traffic inspection, or threat intelligence feeds to further improve detection accuracy.

Challenges remain, particularly in handling false positives. While the system maintains a low false positive rate, misclassifications can still occur, potentially disrupting legitimate financial operations. Continuous model optimization, combined with adaptive alert thresholds, can mitigate these issues. Additionally, system performance depends heavily on the quality and diversity of the training dataset; outdated or biased data could reduce detection effectiveness, emphasizing the importance of ongoing data collection and model retraining.

Balancing security and usability is another consideration. While additional checks and stricter thresholds can enhance security, they may also increase response time. The system achieves a practical balance by providing fast, accurate detection

with clear, actionable alerts. Finally, the modular and scalable design ensures the system can accommodate growing network traffic, making it suitable for deployment in enterprise-level financial networks and adaptable to cloud or hybrid infrastructure.

CHAPTER 7
CONCLUSION AND FUTURE WORK

8.1 Conclusion

This project presents a machine learning-based intrusion detection system designed to enhance network security against increasingly sophisticated cyber-attacks. By utilizing the Random Forest algorithm, the system effectively analyzes network traffic data and accurately classifies activities as normal or malicious. The integration of data preprocessing, feature extraction, and ensemble learning techniques significantly improves detection accuracy while reducing false positive rates. Compared to traditional signature-based systems, the proposed approach demonstrates better adaptability, scalability, and efficiency in handling large and dynamic network environments.

The results confirm that the system is capable of detecting both known and unknown threats with high reliability. Real-time monitoring and alert generation further strengthen the system's ability to respond quickly to potential intrusions. Overall, the project provides a robust and intelligent solution for modern cybersecurity challenges, ensuring better protection of network resources and sensitive information.

8.2 Future Work

Although the proposed system performs effectively, there are several areas for future improvement to further enhance its capabilities. One possible enhancement is the integration of deep learning techniques such as neural networks and LSTM models to detect more complex and evolving attack patterns. These models can improve the system's ability to learn temporal and sequential behaviors in network traffic.

Another enhancement is the development of hybrid ensemble models that combine multiple machine learning algorithms to achieve even higher detection accuracy and robustness. Incorporating real-time streaming data processing frameworks can further improve the system's performance in high-speed network environments. Additionally,

implementing adaptive learning mechanisms will allow the system to continuously update itself based on new attack patterns without manual intervention.

The system can also be extended by integrating with cloud-based platforms and IoT networks to provide scalable and distributed security solutions. Visualization dashboards and user-friendly interfaces can be developed to help administrators better understand network behavior and respond to threats effectively. These enhancements will make the intrusion detection system more intelligent, efficient, and suitable for future cybersecurity demands.

APPENDIX

APPENDIX A : CODE

```
from flask import Flask, render_template, flash, request, session,send_file
from flask import render_template, redirect, url_for, request
#from wtforms import Form, TextField, TextAreaField, validators, StringField,
SubmitField
from werkzeug.utils import secure_filename
import mysql.connector
import matplotlib
import smtplib
#from PIL import Image
import pickle

import numpy as np
import numpy as np
import joblib
from tensorflow.keras.models import load_model
import pandas as pd

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
import joblib




app = Flask(__name__)
```

```
app.config.from_object(__name__)
app.config['SECRET_KEY'] = '7d441f27d441f27567d441f2b6176a'

app.config['DEBUG']
 #Load model and preprocessing tools
model = load_model("lstm_attack_model.h5")
scaler = joblib.load("scaler.pkl")
label_encoder = joblib.load("label_encoder.pkl")
features = joblib.load("features.pkl")

# Load the dataset
df = pd.read_csv("UNSW_NB15_train_set.csv")
df.fillna(0, inplace=True)
df = pd.get_dummies(df, columns=['proto', 'service', 'state'])
df['row_id'] = df.index

@app.route("/")
def homepage():
    return render_template('index.html')
@app.route("/Home")
def Home():
    return render_template('index.html')
@app.route("/AdminLogin")
def AdminLogin():
    return render_template('AdminLogin.html')
@app.route("/NewUser")
def NewUser():
    return render_template('NewUser.html')
@app.route("/UserLogin")
def UserLogin():
    return render_template('UserLogin.html')
```

```
@app.route("/UserHome")
def UserHome():
    return render_template('UserHome.html')
@app.route("/AdminHome")
def AdminHome():
    return render_template('AdminHome.html')

@app.route("/NewQuery1")
def NewQuery1():
    available_ids = df['row_id'].tolist()[:100]
    filled_data = {col: 0 for col in features}
    return render_template('index1.html', features=features, ids=available_ids,
filled_data=filled_data)
    #return render_template('index2.html')
@app.route('/fetch_data', methods=['POST'])
def fetch_data():
    row_id = int(request.form['selected_id'])
    row = df[df['row_id'] == row_id]
    filled_data = {col: row.iloc[0].get(col, 0) for col in features}
    available_ids = df['row_id'].tolist()[:100]
    return render_template('index1.html', features=features, ids=available_ids,
filled_data=filled_data, selected_id=row_id)

@app.route('/download_csv', methods=['POST'])
def download_csv():
    row_id = int(request.form['selected_id'])
    row = df[df['row_id'] == row_id][features]
    file_path = "selected_row.csv"
    row.to_csv(file_path, index=False)
    return send_file(file_path, as_attachment=True)
```

```
@app.route('/predict', methods=['POST'])
def predict():
    try:
        input_data = [float(request.form.get(col, 0)) for col in features]
        filled_data = {col: request.form.get(col, 0) for col in features}
        X_scaled = scaler.transform([input_data]).reshape(1, len(features), 1)
        prediction = model.predict(X_scaled)
        predicted_class = label_encoder.inverse_transform([np.argmax(prediction)])
        available_ids = df['row_id'].tolist()[:100]
        return render_template('index1.html', features=features, ids=available_ids,
                               prediction_text=f"Predicted Attack Category:
{predicted_class[0]}",
                               filled_data=filled_data)
    except Exception as e:
        return f"Error: {e}"




@app.route("/UploadDataset")
def UploadDataset():
    return render_template('ViewExcel.html')
@app.route("/adminlogin", methods=['GET', 'POST'])
def adminlogin():
    error = None
    if request.method == 'POST':
       if request.form['uname'] == 'admin' or request.form['password'] == 'admin':
          conn = mysql.connector.connect(user='root', password='', host='localhost',
database='1heartdb')
          cursor = conn.cursor()
```

```
            cur = conn.cursor()
            cur.execute("SELECT * FROM register")
            data = cur.fetchall()
            return render_template('AdminHome.html', data=data)

        else:
         return render_template('index.html', error=error)
```

```
@app.route("/reg", methods=['GET', 'POST'])
def reg():
    if request.method == 'POST':
        n = request.form['name']

        address = request.form['address']
        age = request.form['age']
        pnumber = request.form['phone']
        email = request.form['email']
        zip = request.form['zip']
        uname = request.form['uname']
        password = request.form['psw']
        conn = mysql.connector.connect(user='root', password='', host='localhost',
```

```
database='1heartdb')
        cursor = conn.cursor()
        cursor.execute(
            "INSERT INTO register VALUES ('" + n + "','" + age + "','" + email + "','" +
pnumber + "','" + zip + "','" + address + "','" + uname + "','" + password + "')")
        conn.commit()
        conn.close()
        # return 'file register successfully'
        return render_template('UserLogin.html')
```

```
@app.route("/userlogin", methods=['GET', 'POST'])
def userlogin():
    error = None
    if request.method == 'POST':
        username = request.form['uname']
        password = request.form['password']
        session['uname'] = request.form['uname']

        conn = mysql.connector.connect(user='root', password='', host='localhost',
database='1heartdb')
        cursor = conn.cursor()
        cursor.execute("SELECT * from register where uname='" + username + "' and
psw='" + password + "'")
        data = cursor.fetchone()
        if data is None:
            return render_template('index.html')
```

```
            return 'Username or Password is wrong'
        else:
            print(data[0])
            session['uid'] = data[0]
            conn = mysql.connector.connect(user='root', password='', host='localhost',
database='1heartdb')
            # cursor = conn.cursor()
            cur = conn.cursor()
            cur.execute("SELECT * FROM register where uname='" + username + "' and
psw='" + password + "'")
            data = cur.fetchall()

            return render_template('UserHome.html', data=data )
@app.route("/UQueryandAns")
def UQueryandAns():

    uname = session['uname']

    conn = mysql.connector.connect(user='root', password='', host='localhost',
database='1heartdb')
    # cursor = conn.cursor()
    cur = conn.cursor()
    cur.execute("SELECT * FROM Querytb1 where UserName='" + uname + "'")
    data = cur.fetchall()

    conn = mysql.connector.connect(user='root', password='', host='localhost',
database='1heartdb')
    # cursor = conn.cursor()
    cur = conn.cursor()
    cur.execute("SELECT * FROM Querytb1 where UserName='" + uname + "'")
    data1 = cur.fetchall()
```

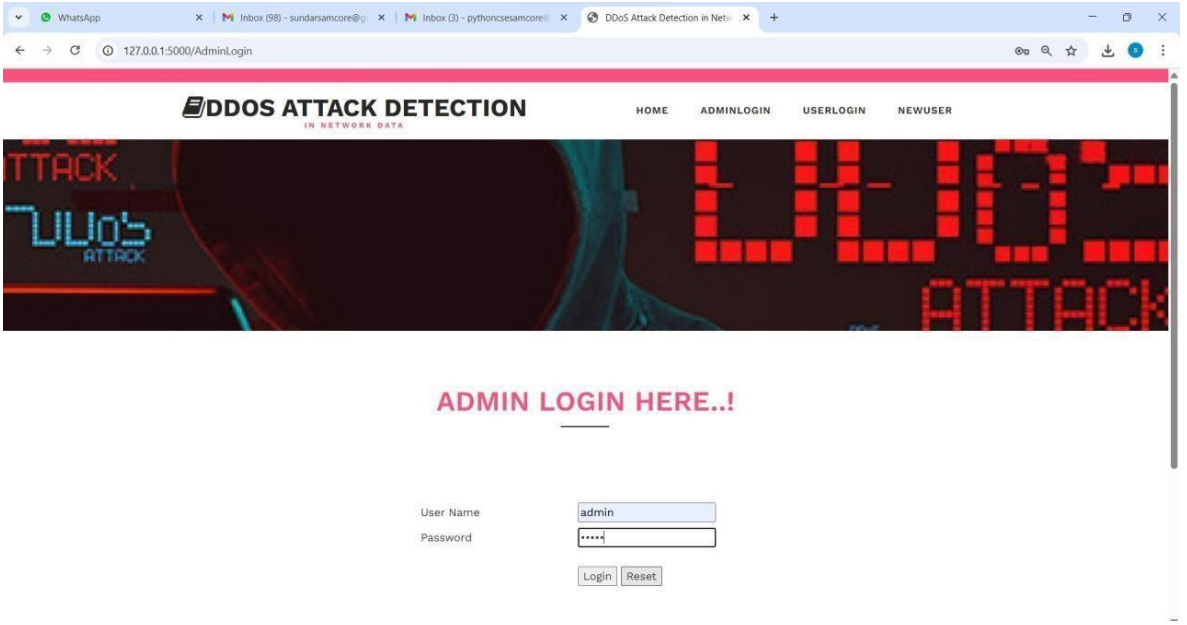
```
    return render_template('UserQueryInfo.html', wait=data, answ=data1 )

@app.route("/AdminQinfo")
def AdminQinfo():

    #uname = session['uname']

    conn = mysql.connector.connect(user='root', password='', host='localhost',
database='1heartdb')
    # cursor = conn.cursor()
    cur = conn.cursor()
    cur.execute("SELECT * FROM Querytb1")
    data = cur.fetchall()
    return render_template('AdminAnswer.html', data=data )
@app.route("/AdminAinfo")
def AdminAinfo():
    conn = mysql.connector.connect(user='root', password='', host='localhost',
database='1heartdb')
    # cursor = conn.cursor()
    cur = conn.cursor()
    cur.execute("SELECT * FROM Querytb1 where DResult !='waiting'")
    data = cur.fetchall()
    return render_template('AdminAnswer.html', data=data )
@app.route("/excelpost", methods=['GET', 'POST'])
def uploadassign():
    if request.method == 'POST':
        file = request.files['fileupload']
        file_extension = file.filename.split('.')[1]
        print(file_extension)
```

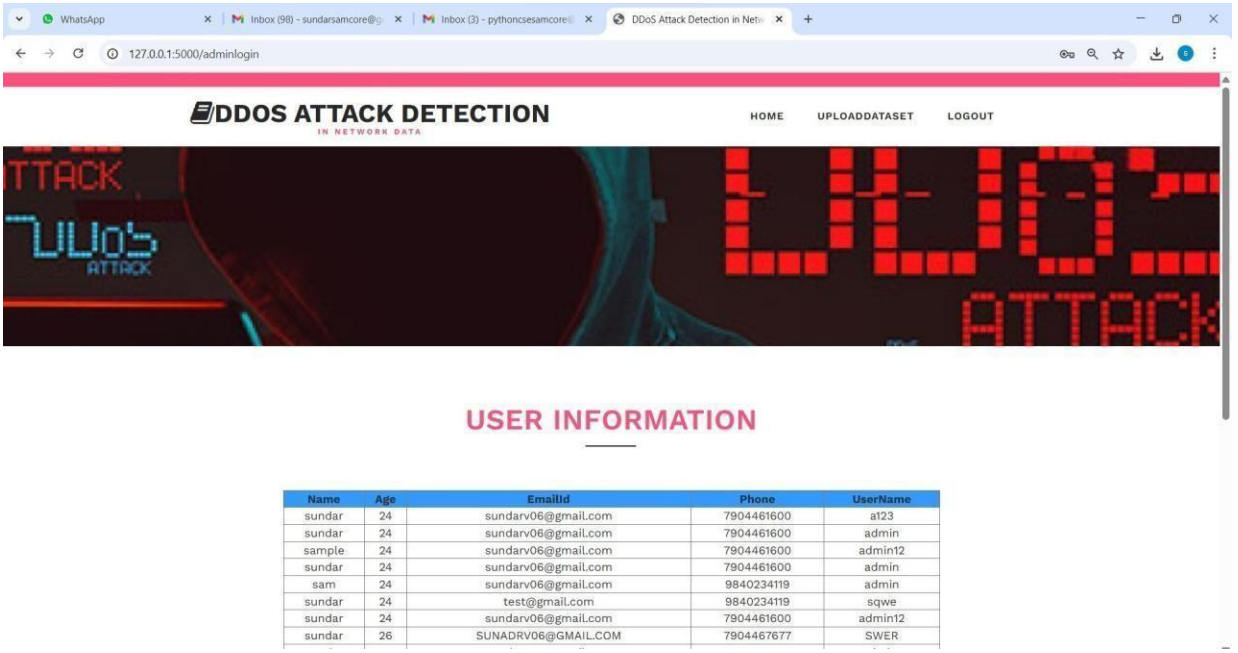
```
        #file.save("static/upload/" + secure_filename(file.filename))

        import pandas as pd
        import matplotlib.pyplot as plt
        df = ''
        if file_extension == 'xlsx':
            df = pd.read_excel(file.read(), engine='openpyxl')
        elif file_extension == 'xls':
            df = pd.read_excel(file.read())
        elif file_extension == 'csv':
            df = pd.read_csv(file)

        print(df)

        import seaborn as sns
        sns.countplot(df['label'], label="Count")
        plt.savefig('static/images/out.jpg')
        iimg = 'static/images/out.jpg'

        plt.show()
        import model
import pandas as pd
import numpy as np
import joblib
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.utils import to_categorical

# Load dataset
```

```
df = pd.read_csv("UNSW_NB15_train_set.csv")
df['attack_cat'] = df['attack_cat'].fillna('Normal')

# One-hot encode categorical features
df = pd.get_dummies(df, columns=['proto', 'service', 'state'])

# Encode target labels
le = LabelEncoder()
df['attack_cat_encoded'] = le.fit_transform(df['attack_cat'])
y = to_categorical(df['attack_cat_encoded'])

# Feature selection
X = df.drop(['label', 'attack_cat', 'attack_cat_encoded'], axis=1)
features = X.columns.tolist()
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = X_scaled.reshape(X_scaled.shape[0], X_scaled.shape[1], 1)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2,
random_state=42)

# LSTM model
model = Sequential()
model.add(LSTM(64, input_shape=(X_train.shape[1], 1)))
model.add(Dropout(0.3))
model.add(Dense(32, activation='relu'))
model.add(Dense(y.shape[1], activation='softmax'))

model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
```

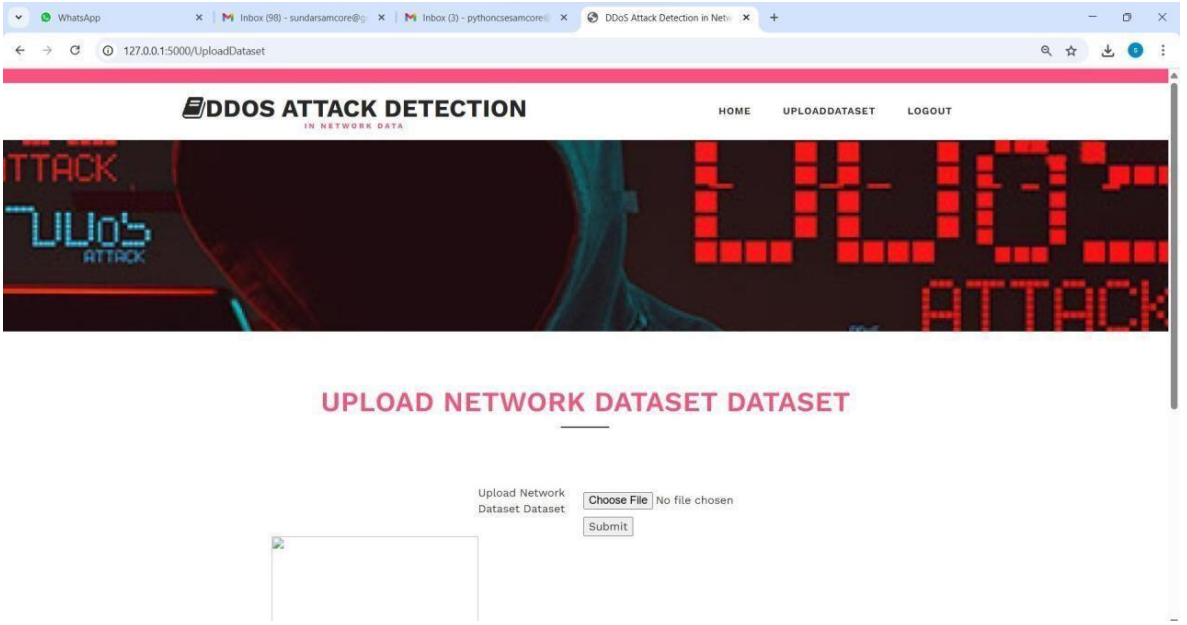
```
model.fit(X_train, y_train, epochs=10, batch_size=128, validation_data=(X_test,
y_test))

# Save model and preprocessors
model.save("lstm_attack_model.h5")
joblib.dump(scaler, "scaler.pkl")
joblib.dump(le, "label_encoder.pkl")
joblib.dump(features, "features.pkl")

print("LSTM model trained and saved.")



        return render_template('ViewExcel.html', data=df.to_html(), dataimg=iimg)

if __name___== '__main__':
    app.run(debug=True, use_reloader=True)
```

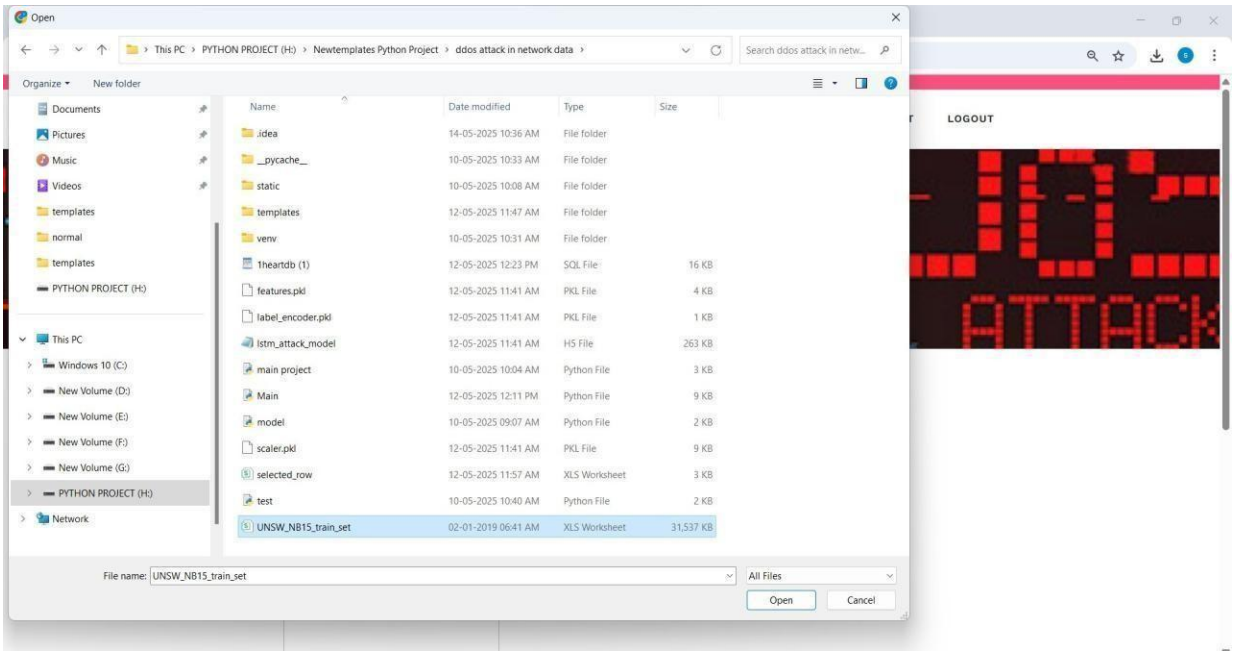
APPENDIX B : OUTPUT SCREENSHOTS

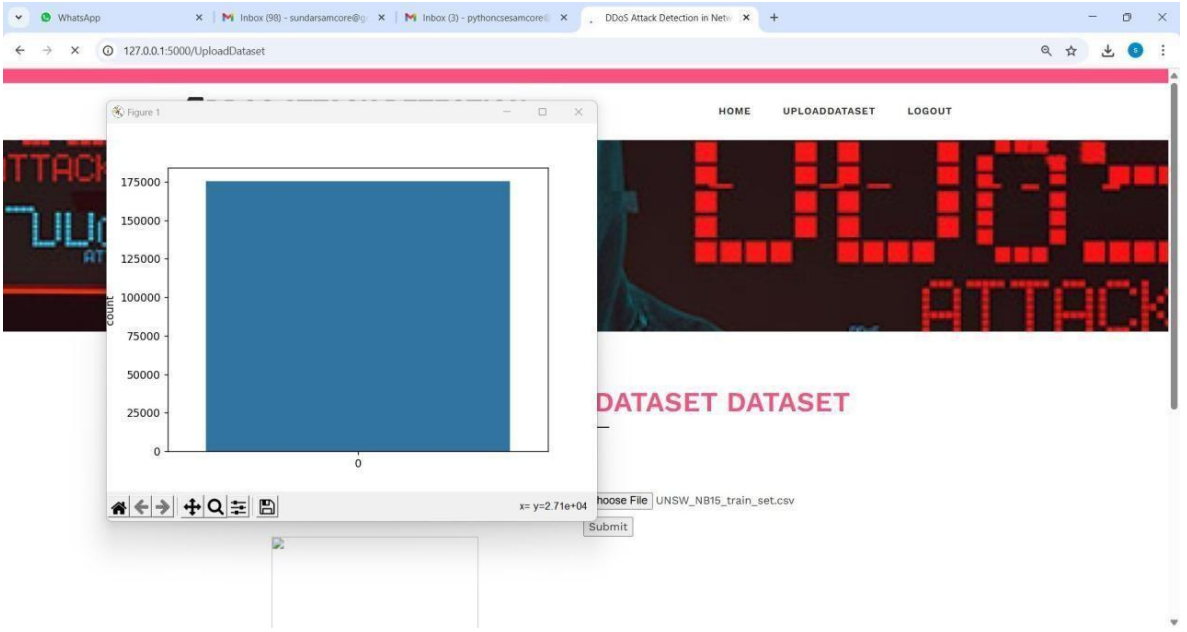


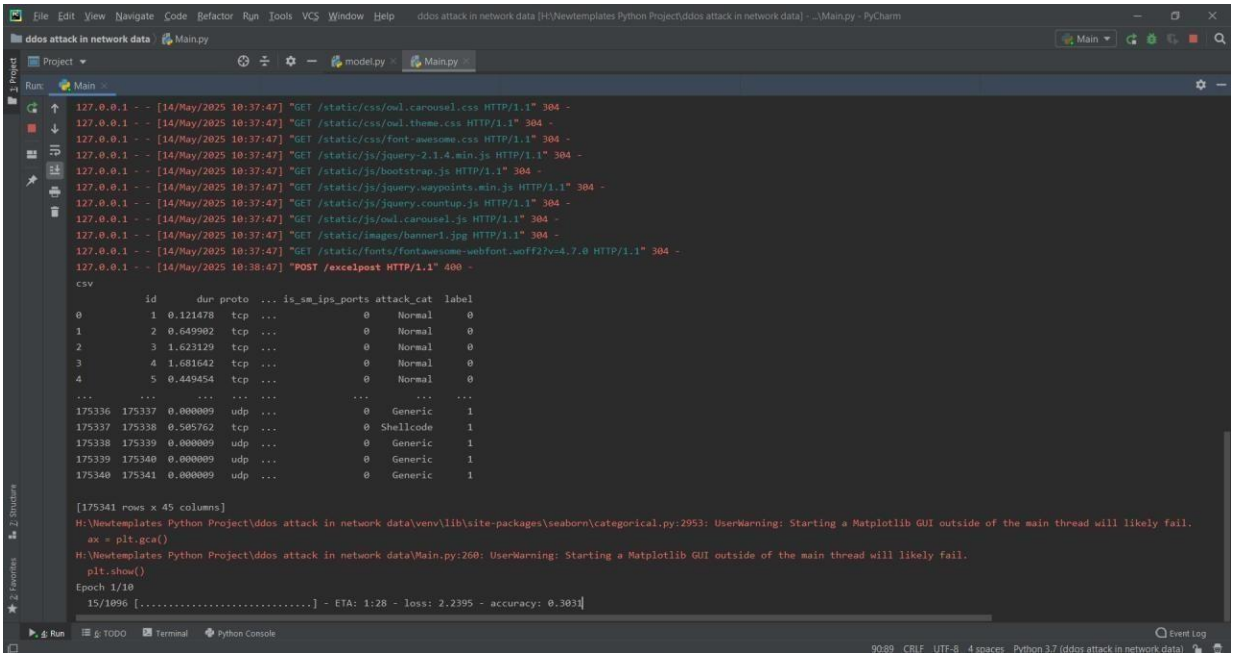


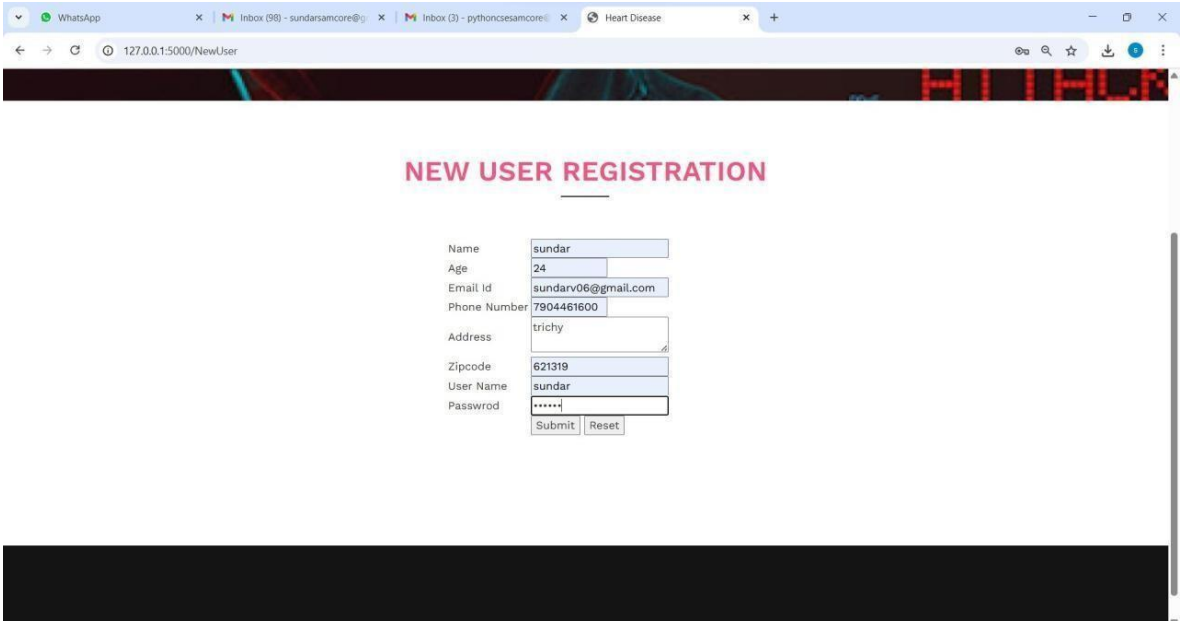
Name	Age	EmailId	Phone	UserName
sundar	24	sundarv06@gmail.com	7904461600	a123
sundar	24	sundarv06@gmail.com	7904461600	admin
sample	24	sundarv06@gmail.com	7904461600	admin12
sundar	24	sundarv06@gmail.com	7904461600	admin
sam	24	sundarv06@gmail.com	9840234119	admin
sundar	24	test@gmail.com	9840234119	sqwe
sundar	24	sundarv06@gmail.com	7904461600	admin12
sundar	26	SUNADRV06@GMAIL.COM	7904467677	SWER

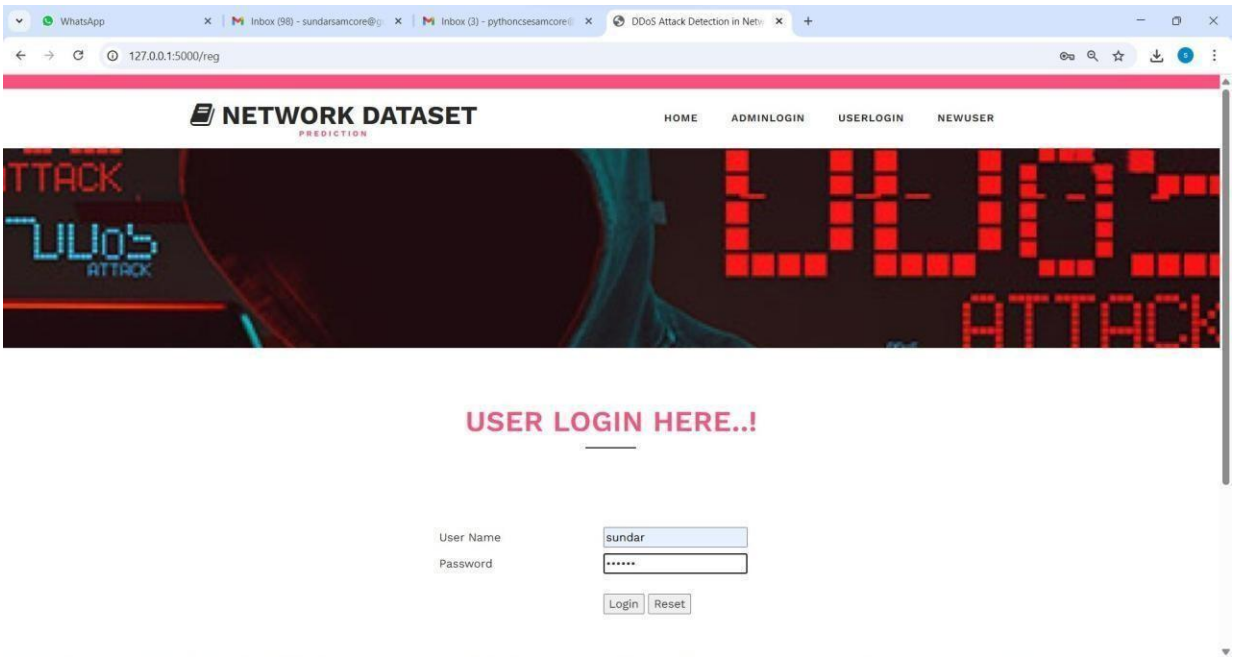


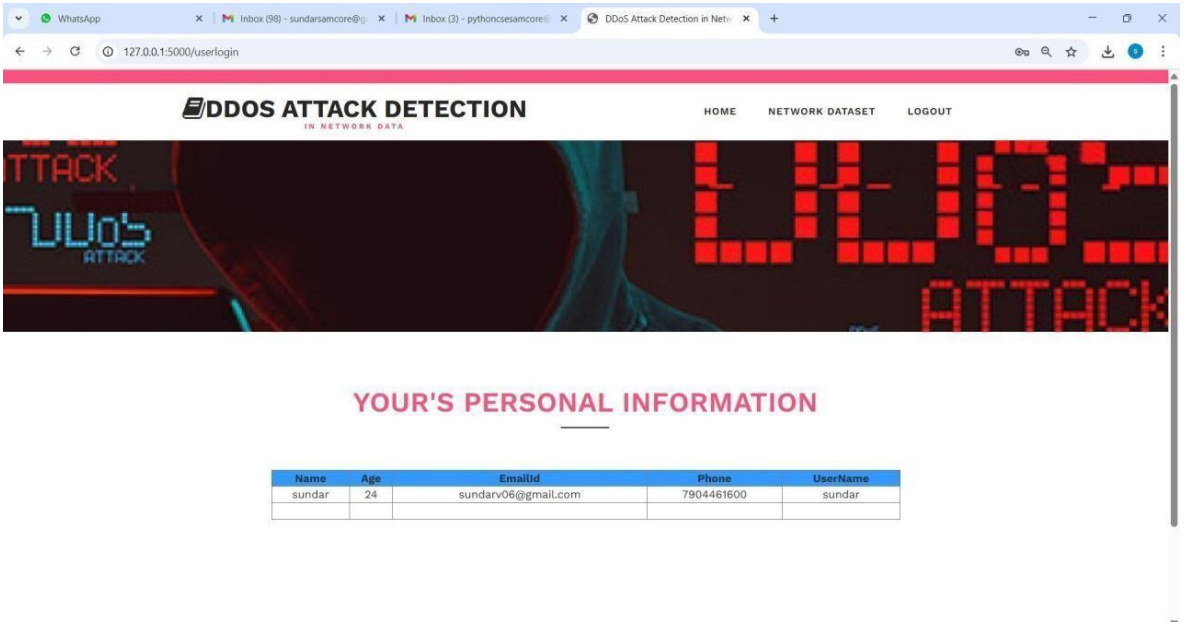

Open
This PC › PYTHON PROJECT (H:) › Newtemplates Python Project › ddos attack in network data ›
Search ddos attack in netw...
Organize
New folder
Name
Date modified
Type
Size
.idea 14-05-2025 10:36 AM File folder
__pycache__ 10-05-2025 10:33 AM File folder
static 10-05-2025 10:08 AM File folder
templates 12-05-2025 11:47 AM File folder
venv 10-05-2025 10:31 AM File folder
1heartdb (1) 12-05-2025 12:23 PM SQL File 16 KB
features.pkl 12-05-2025 11:41 AM PKL File 4 KB
label_encoder.pkl 12-05-2025 11:41 AM PKL File 1 KB
lstm_attack_model 12-05-2025 11:41 AM H5 File 263 KB
main project 10-05-2025 10:04 AM Python File 3 KB
Main 12-05-2025 12:11 PM Python File 9 KB
model 10-05-2025 09:07 AM Python File 2 KB
scaler.pkl 12-05-2025 11:41 AM PKL File 9 KB
selected_row 12-05-2025 11:57 AM XLS Worksheet 3 KB
test 10-05-2025 10:40 AM Python File 2 KB
UNSW_NB15_train_set 02-01-2019 06:41 AM XLS Worksheet 31,537 KB
Documents
Pictures
Music
Videos
templates
normal
templates
PYTHON PROJECT (H:)
This PC
Windows 10 (C:)
New Volume (D:)
New Volume (E:)
New Volume (F:)
New Volume (G:)
PYTHON PROJECT (H:)
Network
File name: UNSW_NB15_train_set
All Files
Open
Cancel
LOGOUT
ATTACK

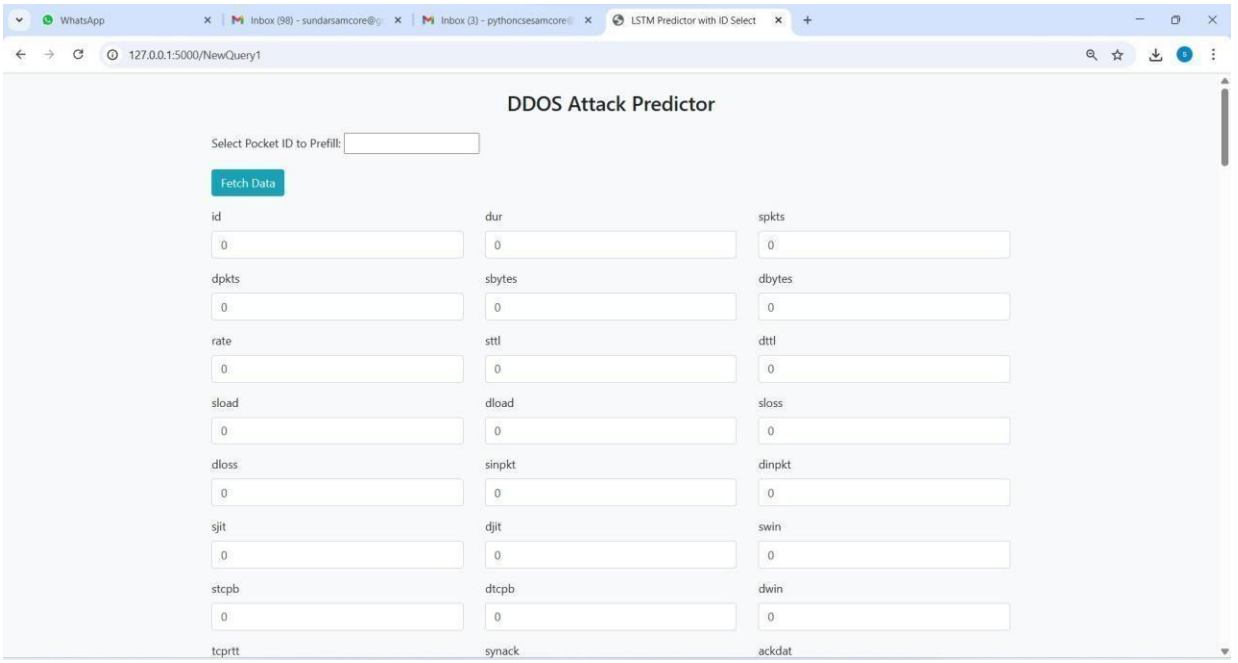

WhatsApp
Inbox (98) - sundarsamcore@g
Inbox (3) - pythoncsesamcore@
DDoS Attack Detection in Netw
127.0.0.1:5000/UploadDataset
HOME
UPLOADDATASET
LOGOUT
Figure 1
175000
150000
125000
100000
75000
50000
25000
0
count
0
x= y=2.71e+04
ATTACK
DATASET DATASET
hoose File
UNSW_NB15_train_set.csv
Submit

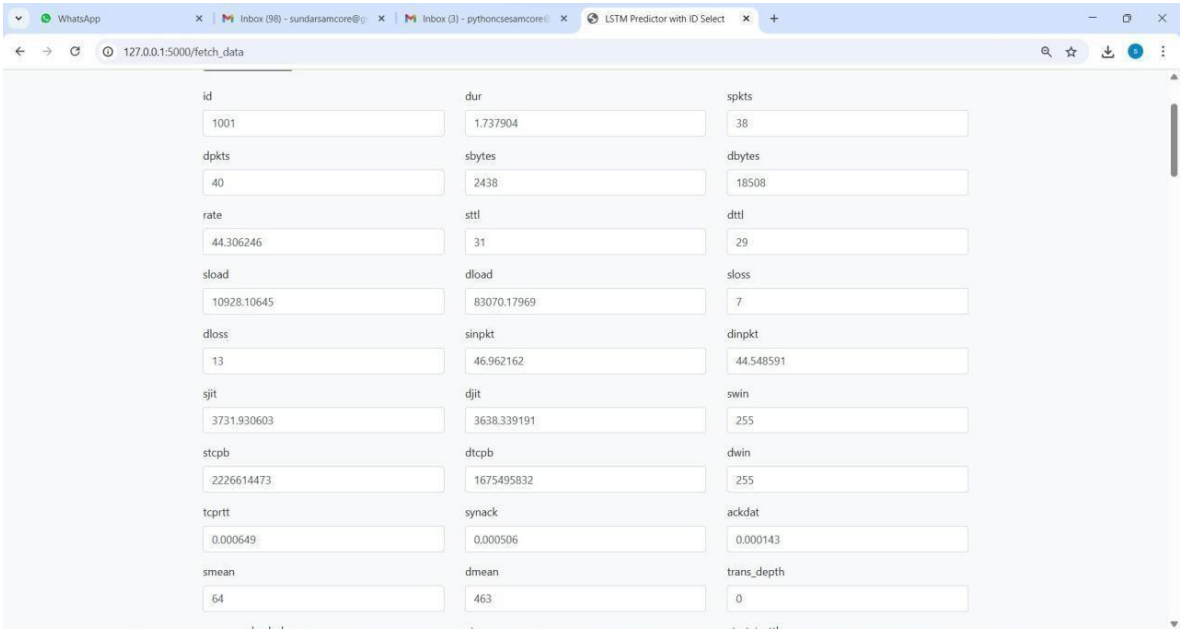

ddos attack in network data
Main.py
model.py
Main.py
Run: Main
127.0.0.1 - - [14/May/2025 10:37:47] "GET /static/css/owl.carousel.css HTTP/1.1" 304 -
127.0.0.1 - - [14/May/2025 10:37:47] "GET /static/css/owl.theme.css HTTP/1.1" 304 -
127.0.0.1 - - [14/May/2025 10:37:47] "GET /static/css/font-awesome.css HTTP/1.1" 304 -
127.0.0.1 - - [14/May/2025 10:37:47] "GET /static/js/jquery-2.1.4.min.js HTTP/1.1" 304 -
127.0.0.1 - - [14/May/2025 10:37:47] "GET /static/js/bootstrap.js HTTP/1.1" 304 -
127.0.0.1 - - [14/May/2025 10:37:47] "GET /static/js/jquery.waypoints.min.js HTTP/1.1" 304 -
127.0.0.1 - - [14/May/2025 10:37:47] "GET /static/js/jquery.countup.js HTTP/1.1" 304 -
127.0.0.1 - - [14/May/2025 10:37:47] "GET /static/js/owl.carousel.js HTTP/1.1" 304 -
127.0.0.1 - - [14/May/2025 10:37:47] "GET /static/images/banner1.jpg HTTP/1.1" 304 -
127.0.0.1 - - [14/May/2025 10:37:47] "GET /static/fonts/fontawesome-webfont.woff2?v=4.7.0 HTTP/1.1" 304 -
127.0.0.1 - - [14/May/2025 10:38:47] "POST /excelpost HTTP/1.1" 400 -
csv
id dur proto ... is_sm_ips_ports attack_cat label
0 1 0.121478 tcp ... 0 Normal 0
1 2 0.649902 tcp ... 0 Normal 0
2 3 1.623129 tcp ... 0 Normal 0
3 4 1.681642 tcp ... 0 Normal 0
4 5 0.449454 tcp ... 0 Normal 0
175336 175337 0.000009 udp ... 0 Generic 1
175337 175338 0.505762 tcp ... 0 Shellcode 1
175338 175339 0.000009 udp ... 0 Generic 1
175339 175340 0.000009 udp ... 0 Generic 1
175340 175341 0.000009 udp ... 0 Generic 1
[175341 rows x 45 columns]
H:\Newtemplates Python Project\ddos attack in network data\venv\lib\site-packages\seaborn\categorical.py:2953: UserWarning: Starting a Matplotlib GUI outside of the main thread will likely fail.
ax = plt.gca()
H:\Newtemplates Python Project\ddos attack in network data\Main.py:260: UserWarning: Starting a Matplotlib GUI outside of the main thread will likely fail.
plt.show()
Epoch 1/10
15/1096 [.....] - ETA: 1:28 - loss: 2.2395 - accuracy: 0.3031
Run
TODO
Terminal
Python Console
Event Log

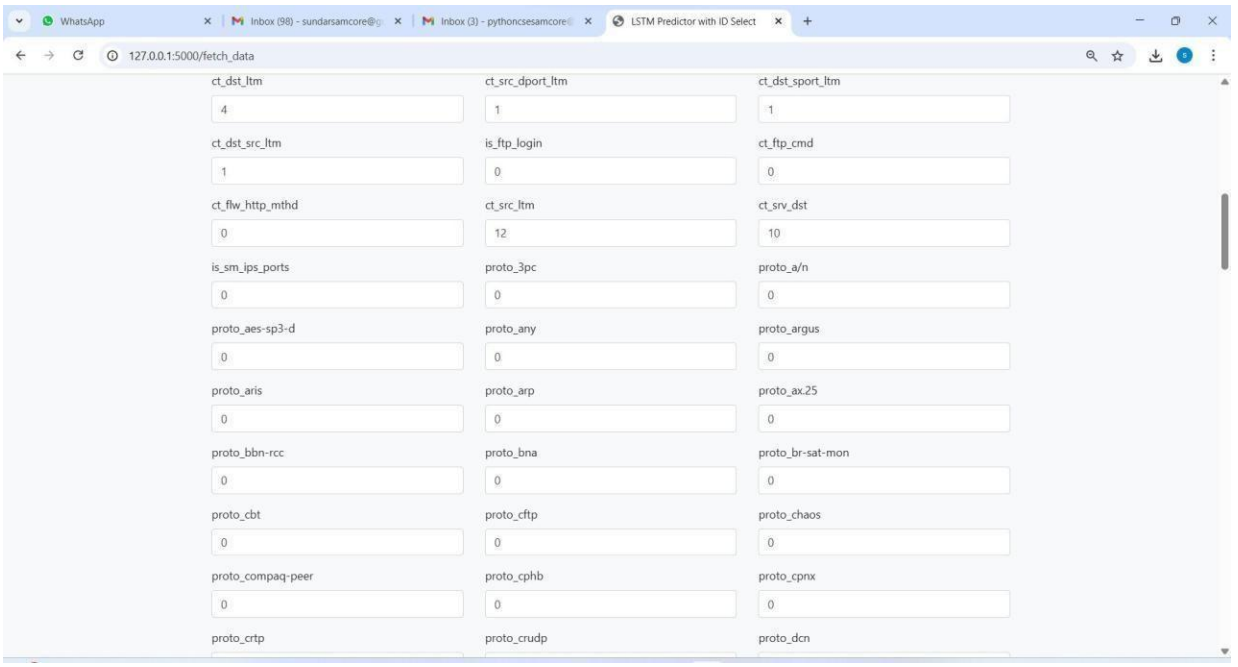

WhatsApp
Inbox (98) - sundarsamcore@g
Inbox (3) - pythoncsesamcore
Heart Disease
127.0.0.1:5000/NewUser
NEW USER REGISTRATION
Name sundar
Age 24
Email Id sundarv06@gmail.com
Phone Number 7904461600
Address trichy
Zipcode 621319
User Name sundar
Passwrod
Submit
Reset

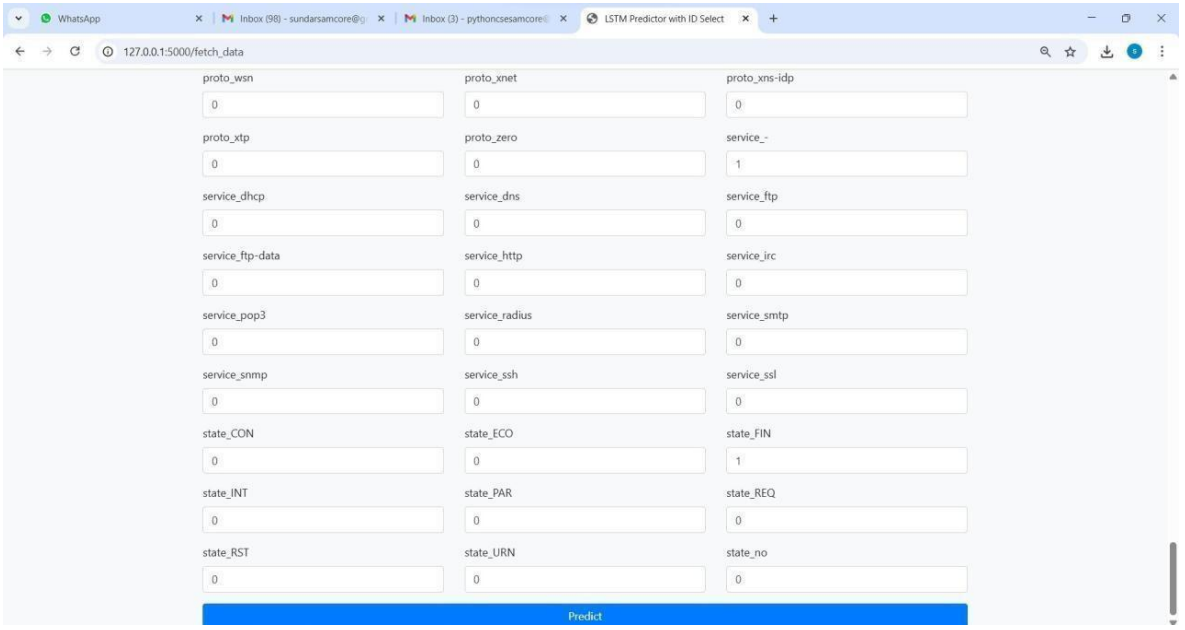

WhatsApp
Inbox (98) - sundarsamcore@g
Inbox (3) - pythoncsesamcore
DDoS Attack Detection in Netw
127.0.0.1:5000/reg
NETWORK DATASET
PREDICTION
HOME
ADMINLOGIN
USERLOGIN
NEWUSER
USER LOGIN HERE..!
User Name
sundar
Password
Login
Reset

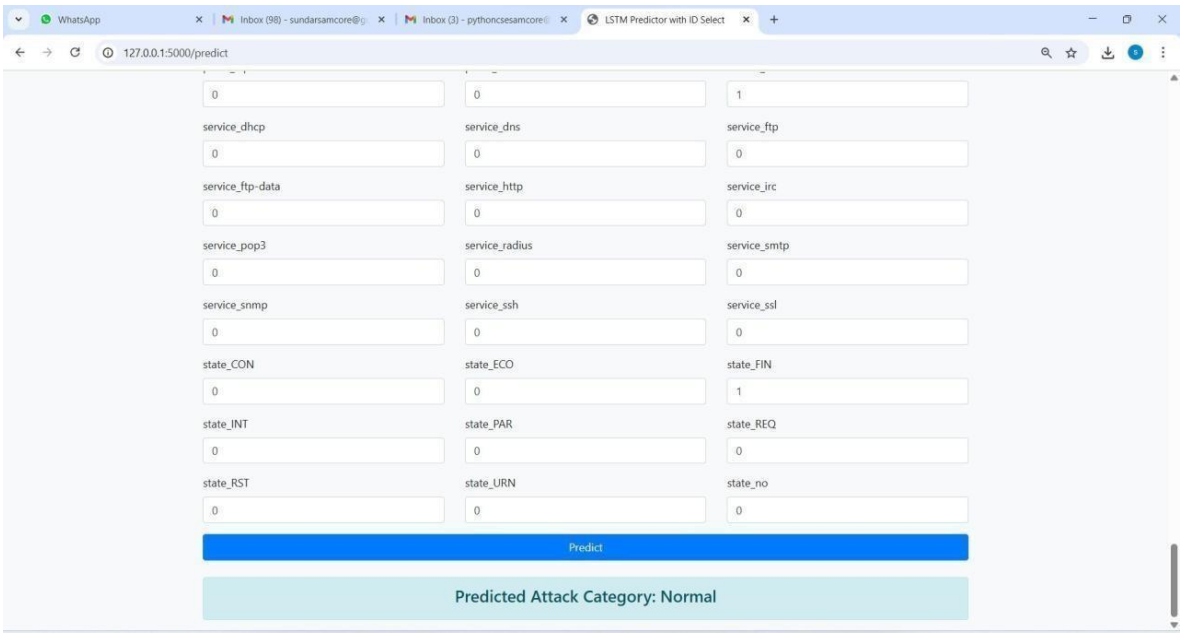

WhatsApp
Inbox (98) - sundarsamcore@g
Inbox (3) - pythoncsesamcore
DDoS Attack Detection in Netw
127.0.0.1:5000/userlogin
DDOS ATTACK DETECTION
IN NETWORK DATA
HOME
NETWORK DATASET
LOGOUT
YOUR'S PERSONAL INFORMATION
Name
Age
EmailId
Phone
UserName
sundar
24
sundarv06@gmail.com
7904461600
sundar


WhatsApp
Inbox (98) - sundarsamcore@g
Inbox (3) - pythoncsesamcore@
LSTM Predictor with ID Select
127.0.0.1:5000/NewQuery1
DDOS Attack Predictor
Select Pocket ID to Prefill:
Fetch Data
id
dur
spkts
dpkts
sbytes
dbytes
rate
sttl
dttl
sload
dload
sloss
dloss
sinpkt
dinpkt
sjit
djit
swin
stcpb
dtcpb
dwin
tcprtt
synack
ackdat


WhatsApp
Inbox (98) - sundarsamcore@g
Inbox (3) - pythoncsesamcore@
LSTM Predictor with ID Select
127.0.0.1:5000/fetch_data
id
1001
dur
1.737904
spkts
38
dpkts
40
sbytes
2438
dbytes
18508
rate
44.306246
sttl
31
dttl
29
sload
10928.10645
dload
83070.17969
sloss
7
dloss
13
sinpkt
46.962162
dinpkt
44.548591
sjit
3731.930603
djit
3638.339191
swin
255
stcpb
2226614473
dtcpb
1675495832
dwin
255
tcprtt
0.000649
synack
0.000506
ackdat
0.000143
smean
64
dmean
463
trans_depth
0


WhatsApp
Inbox (98) - sundarsamcore@g
Inbox (3) - pythoncsesamcore
LSTM Predictor with ID Select
127.0.0.1:5000/fetch_data
ct_dst_ltm
4
ct_src_dport_ltm
1
ct_dst_sport_ltm
1
ct_dst_src_ltm
1
is_ftp_login
0
ct_ftp_cmd
0
ct_flw_http_mthd
0
ct_src_ltm
12
ct_srv_dst
10
is_sm_ips_ports
0
proto_3pc
0
proto_a/n
0
proto_aes-sp3-d
0
proto_any
0
proto_argus
0
proto_aris
0
proto_arp
0
proto_ax.25
0
proto_bbn-rcc
0
proto_bna
0
proto_br-sat-mon
0
proto_cbt
0
proto_cftp
0
proto_chaos
0
proto_compaq-peer
0
proto_cphb
0
proto_cpnx
0
proto_crtp
proto_crudp
proto_dcn


WhatsApp
Inbox (98) - sundarsamcore@g
Inbox (3) - pythoncsesamcore
LSTM Predictor with ID Select
127.0.0.1:5000/fetch_data
proto_wsn
0
proto_xnet
0
proto_xns-idp
0
proto_xtp
0
proto_zero
0
service_-
1
service_dhcp
0
service_dns
0
service_ftp
0
service_ftp-data
0
service_http
0
service_irc
0
service_pop3
0
service_radius
0
service_smtp
0
service_snmp
0
service_ssh
0
service_ssl
0
state_CON
0
state_ECO
0
state_FIN
1
state_INT
0
state_PAR
0
state_REQ
0
state_RST
0
state_URN
0
state_no
0
Predict


WhatsApp
Inbox (98) - sundarsamcore@g
Inbox (3) - pythoncsesamcore@
LSTM Predictor with ID Select
127.0.0.1:5000/predict
0
0
1
service_dhcp
service_dns
service_ftp
0
0
0
service_ftp-data
service_http
service_irc
0
0
0
service_pop3
service_radius
service_smtp
0
0
0
service_snmp
service_ssh
service_ssl
0
0
0
state_CON
state_ECO
state_FIN
0
0
1
state_INT
state_PAR
state_REQ
0
0
0
state_RST
state_URN
state_no
0
0
0
Predict
Predicted Attack Category: Normal

REFERENCE

1. Saraswathi, V., and R. Dayana. "Enhancing security in next generation networks: A deep learning approach for intrusion detection." 2025 4th International Conference on Sentiment Analysis and Deep Learning (ICSADL). IEEE, 2025.
2. Bankó, Márton Bendegúz, et al. "Advancements in machine learning-based intrusion detection in iot: Research trends and challenges." Algorithms 18.4 (2025): 209.
3. Raja, Muhammadu Sathik Raja Sathik. "The rise of ai-driven network intrusion detection systems: Innovations, challenges, and future directions." International Journal of AI, BigData, Computational and Management Studies 6.1 (2025): 1-9.
4. Abou-Elasaad, Mounir Mohammad, Samir G. Sayed, and Mohamed M. El-Dakroury. "Securing the Future: Real-Time intrusion Detection in IIoT Smart Grids through Innovative AI Solutions." Journal of Cybersecurity & Information Management 15.2 (2025).
5. Amine, Marina S., Fayza A. Nada, and Khalid M. Hosny. "Improved model for intrusion detection in the internet of things." Scientific Reports 15.1 (2025): 21547.
6. Mukkamala, S., Janoski, G., & Sung, A. (2023). A deep learning approach for network intrusion detection system. IEEE. In Proceedings of the 2023 International Joint Conference on Neural Networks (IJCNN02 Cat No 02 CH3 7290).
7. Haijun, X., Fang, P., Ling, W., & Hongwei, L. (2022). A deep learning approach for intrusion detection using recurrent neural networks (RNN). IEEE. In Proceedings of Grey Systems and Intelligent Services (GSIS 07).
8. Yu, Y. (2023). Network intrusion detection based on deep learning. ACM Journal of Computing Sciences in Colleges, 25(5).

9. Tuba, M., & Bulatovic, D. (2022). A deep learning approach for intrusion detection using recurrent neural networks (RNN). IEEE Transactions.
10. Khan, N. Y., Rauf, B., & Ahmed, K. (2021). Deep learning for intrusion detection systems. IEEE.

Stay Ahead

BANNARI AMMAN INSTITUTE OF TECHNOLOGY

(Autonomous Institution Affiliated to Anna University Chennai Approved by AICTE-Accredited by NAAC with 'A+' Grade)
Sathyamangalam – 638401 Erode (Dt) Tamilnadu

SSMA BIT

CERTIFICATE OF PRESENTATION

This is to Certifiy that Dr./Mr./Ms. SALAMAN CHRISTOPHER Y From

JAYALAKSHMI INSTITUTE OF TECHNOLOGY Presented the Paper titled

INTELLIGENT NETWORK INTRUSION DETECTION SYSTEM USING DEEP LEARNING FOR REAL-TIME THREAT DETECTION

in the National Conference on Sustainable Smart Manufacturing and Automation (SSMA 2026) Organized by the Department of Mechatronics Engineering, Bannari Amman Institute of Technology, Sathyamangalam on 24 March 2026.

Dr V Vadivel Vivek
Organizing Secretary

Dr K L Senthil Kumar
Convenor

Dr C Palanisamy
Principal

Certificate ID : SSMA 304

Stay Ahead

BANNARI AMMAN INSTITUTE OF TECHNOLOGY

(Autonomous Institution Affiliated to Anna University Chennai Approved by AICTE-Accredited by NAAC with 'A+' Grade)
Sathyamangalam – 638401 Erode (Dt) Tamilnadu

SSMA BIT

CERTIFICATE OF PRESENTATION

This is to Certifiy that Dr./Mr./Ms. RAMYA A From

JAYALAKSHMI INSTITUTE OF TECHNOLOGY Presented the Paper titled

INTELLIGENT NETWORK INTRUSION DETECTION SYSTEM USING DEEP LEARNING FOR REAL-TIME THREAT DETECTION

in the National Conference on Sustainable Smart Manufacturing and Automation (SSMA 2026) Organized by the Department of Mechatronics Engineering, Bannari Amman Institute of Technology, Sathyamangalam on 24 March 2026.

Dr V Vadivel Vivek
Organizing Secretary

Dr K L Senthil Kumar
Convenor

Dr C Palanisamy
Principal

Certificate ID : SSMA 304

Stay Ahead

BANNARI AMMAN INSTITUTE OF TECHNOLOGY

(Autonomous Institution Affiliated to Anna University Chennai Approved by AICTE-Accredited by NAAC with 'A+' Grade)
Sathyamangalam – 638401 Erode (Dt) Tamilnadu

SSMA BIT

CERTIFICATE OF PRESENTATION

This is to Certifiy that Dr./Mr./Ms. RAGUNATH CK From

JAYALAKSHMI INSTITUTE OF TECHNOLOGY Presented the Paper titled

INTELLIGENT NETWORK INTRUSION DETECTION SYSTEM USING DEEP LEARNING FOR REAL-TIME THREAT DETECTION

in the National Conference on Sustainable Smart Manufacturing and Automation (SSMA 2026) Organized by the Department of Mechatronics Engineering, Bannari Amman Institute of Technology, Sathyamangalam on 24 March 2026.

Dr V Vadivel Vivek
Organizing Secretary

Dr K L Senthil Kumar
Convenor

Dr C Palanisamy
Principal

Certificate ID : SSMA 304

Stay Ahead

BANNARI AMMAN INSTITUTE OF TECHNOLOGY

(Autonomous Institution Affiliated to Anna University Chennai Approved by AICTE-Accredited by NAAC with 'A+' Grade)
Sathyamangalam – 638401 Erode (Dt) Tamilnadu

SSMA BIT

CERTIFICATE OF PRESENTATION

This is to Certifiy that Dr./Mr./Ms. SATHISH KUMAR S From

JAYALAKSHMI INSTITUTE OF TECHNOLOGY Presented the Paper titled

INTELLIGENT NETWORK INTRUSION DETECTION SYSTEM USING DEEP LEARNING FOR REAL-TIME THREAT DETECTION

in the National Conference on Sustainable Smart Manufacturing and Automation (SSMA 2026) Organized by the Department of Mechatronics Engineering, Bannari Amman Institute of Technology, Sathyamangalam on 24 March 2026.

Dr V Vadivel Vivek
Organizing Secretary

Dr K L Senthil Kumar
Convenor

Dr C Palanisamy
Principal

Certificate ID : SSMA 304

Stay Ahead

BANNARI AMMAN INSTITUTE OF TECHNOLOGY

(Autonomous Institution Affiliated to Anna University Chennai Approved by AICTE-Accredited by NAAC with 'A+' Grade)
Sathyamangalam – 638401 Erode (Dt) Tamilnadu

SSMA BIT

CERTIFICATE OF PRESENTATION

This is to Certifiy that Dr./Mr./Ms. VIGNESH V From

JAYALAKSHMI INSTITUTE OF TECHNOLOGY Presented the Paper titled

INTELLIGENT NETWORK INTRUSION DETECTION SYSTEM USING DEEP LEARNING FOR REAL-TIME THREAT DETECTION

in the National Conference on Sustainable Smart Manufacturing and Automation (SSMA 2026) Organized by the Department of Mechatronics Engineering, Bannari Amman Institute of Technology, Sathyamangalam on 24 March 2026.

Dr V Vadivel Vivek
Organizing Secretary

Dr K L Senthil Kumar
Convenor

Dr C Palanisamy
Principal

Certificate ID : SSMA 304